Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Робототехника и комплексная автоматизация»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОМУ ПРОЕКТУ

НА ТЕМУ:

«Проектирование программной части сервера устройства для облачной системы управления складом на основе облачной платформы Yandex.Cloud»

Студент ____РК9-74Б____ _____21.12.2020____ _Д. И. Масный_____
(Группа) (Подпись, дата) (И.О.Фамилия)

Руководитель курсового проекта _____21.12.2020_____ _С. А. Буханов_____
(Подпись, дата) (И.О.Фамилия)

Консультант _____ _М. В. Овсянников_
(Подпись, дата) (И.О.Фамилия)

2020 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой _РК9_____
(Индекс)
_____ _С. С. Гаврюшин_
(И.О.Фамилия)
« _____ » _____ 2020 г.

З А Д А Н И Е
на выполнение курсового проекта

по дисциплине __Основы автоматизации производства_____

Студент группы ____РК9-74Б____

_____Масный Дмитрий Игоревич_____
(Фамилия, имя, отчество)

Тема курсового проекта __Проектирование программной части сервера устройства_____
для облачной системы управления складом на основе облачной платформы_____
Yandex.Cloud_____
_____
_____

Направленность КП (учебный, исследовательский, практический, производственный, др.)
_____учебный_____
Источник тематики (кафедра, предприятие, НИР) _____кафедра_____

График выполнения проекта: 25% к 5 нед., 50% к 9 нед., 75% к 12 нед., 100% к 16 нед.

Задание

1) проанализировать существующее ПО, управляющее складом и выявить его недостатки;
2) на основе анализа полученных данных сформулировать требования, предъявляемые к проектируемой программной части сервера устройства;
3) разработать функциональную модель перемещения паллет складом;
4) разработать алгоритмы основных функций программной части сервера устройства;
5) разработать исходный код программной части сервера устройства на языке C#.

Оформление курсового проекта:

Расчетно-пояснительная записка на 34 листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.):
17 рисунков, 13 слайдов.
Дата выдачи задания «19» сентября 2020 г.

Руководитель курсового проекта _____21.12.2020_____ _С. А. Буханов____
(Подпись, дата) (И.О.Фамилия)

Студент _____21.12.2020_____ _Д. И. Масный____
(Подпись, дата) (И.О.Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

РЕФЕРАТ

Курсовой проект выполнен по теме: «Проектирование программной части сервера устройства для облачной системы управления складом на основе облачной платформы Yandex Cloud».

Целью выполнения курсового проекта является создание программной части сервера устройства для облачной системы управления модулем транспортно-накопительной системы (складом) на основе облачной платформы «Yandex Cloud».

Курсовой проект состоит из расчётно-пояснительной записки, которая содержит 34 листа формата А4, 17 рисунков, 10 используемых источников, 4 приложения; презентации, состоящей из 13 слайдов.

Ключевые слова: сервер устройства, облачная платформа, интернет вещей, промышленный интернет вещей, «Yandex Cloud».

Данная работа состоит из пяти разделов.

В первом разделе приведен анализ существующей системы, по результатам которого были выделены недостатки.

Во втором разделе представлена функциональная модель программы на основе результатов анализа, полученных в первом разделе.

В третьем разделе представлена концептуальная модель предметной области (складской модуль).

В четвертом разделе описано взаимодействие между облачной платформой и сервером устройства, а также между сервером устройства и ПЛК склада.

В пятом разделе разработаны алгоритмы работы основных функций программы, а также графический интерфейс программы, приведены возможные сценарии использования программы пользователем.

СОДЕРЖАНИЕ

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ 6

ВВЕДЕНИЕ 7

1. Анализ существующей системы 10

2. Разработка функциональной модели перемещения паллет складом 13

3. Разработка концептуальной модели 16

4. Взаимодействие склада с облачной платформой «Yandex Cloud» 17

4.1 Взаимодействие сервера устройства с облачной платформой «Yandex Cloud» 18

4.2 Формат сообщений между облачной платформой и сервером устройства 22

4.3 Взаимодействие сервера устройства с ПЛК склада 23

4.4 Использование библиотеки «Mint Motion Library» 24

5. Разработка программной части сервера устройства 27

5.1 Проектирование алгоритма подключения сервера устройства к облачной платформе «Yandex Cloud» 27

5.2 Проектирование алгоритма функции валидации полученных команд от облачной платформы «Yandex Cloud» 29

5.3 Проектирование алгоритма функции перемещения штабелера 30

5.4 Разработка графического интерфейса программы 32

ЗАКЛЮЧЕНИЕ 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 36

Приложение А 37

Приложение Б 39

Приложение В 43

Приложение Г 67

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

«IOT» («Internet of Things») – система взаимосвязанных устройств (вещей), оснащенных средствами для беспроводной передачи данных между друг другом, внешней средой без непосредственного вмешательства человека.

«IIOT» («Industrial Internet of Things») – взаимосвязанные датчики, исполнительные устройства, промышленное оборудование и другие девайсы, объединенные в сеть совместно с промышленным программным обеспечением для управления производством.

«M2M» («Machine-to-Machine») – общее название технологий, которые позволяют машинам обмениваться информацией друг с другом, или же передавать её в одностороннем порядке.

«MML» («Mint Motion Library») – библиотека функций, разработанная компанией «ABB», позволяющая пользователю управлять промышленным оборудованием напрямую с ПК.

«Yandex Cloud» – облачная платформа от российской компании «Yandex», используемая в данном курсовом проекте.

«Mint ActiveX» – объект, который встраивается в программу, для того чтобы предоставить пользователю возможность управлять промышленным оборудованием с помощью приложений на базе ПК, взаимодействующих с ПЛК.

«MIL-сервер» – программный компонент, предоставляемый компанией ABB, который отвечает за передачу вызовов функции библиотеки «MML» между пользовательским приложением и ПЛК.

«MQTT» («Message Queuing Telemetry Transport») – упрощённый сетевой протокол, работающий поверх TCP/IP, ориентированный на обмен сообщений между устройствами по принципу издатель-подписчик.

«MQTT-брокер» – сервер-посредник, используемый в протоколе MQTT.

ВВЕДЕНИЕ

В настоящее время мир находится в состоянии четвертой промышленной революции, которая приводит к изменениям не только в социальной, политической, экономической сферах жизни, но и в промышленности. Одним из аспектов данного технологического прогресса является использование кибер-физических систем и технологии интернета вещей («IoT») в промышленности.

Кибер-физическая система – это интеграция вычислительных ресурсов в физическую сущность любого вида [3]. Внедрение кибер-физических систем в производство принято называть индустрия 4.0, его основная цель – максимальная автоматизация производства, частичное или полное исключение человека из производственных процессов. Такие системы позволяют улучшить производственные процессы, обеспечивая обмен данными между промышленным оборудованием в режиме реального времени, который возможен только в локальной сети, например, в пределах одной ГПЯ.

Для того чтобы создать умное производство, необходимо также применять технологии интернета вещей, которые позволяют в автоматическом режиме вести мониторинг оборудования, контролировать производственный процесс не только в рамках одного производственного модуля, но и в рамках всего предприятия в целом [11].

Термин «интернет вещей» был впервые использован Кэвином Энтоном, основателем исследовательской группы «Auto-ID» при Массачусетском технологическом университете («MIT»), во время презентации для руководства компании «Procter&Gamble» в 1999 году [8]. С тех пор было представлено множество определений данного понятия, но в настоящее время общепринятым является следующее: интернет вещей («Internet of Things») – система взаимосвязанных устройств (вещей), оснащенных средствами для беспроводной передачи данных между друг другом, внешней средой без непосредственного вмешательства человека. Развитие данной технологии привело к появлению

такого направления, как интернет всего («Internet of Everything») – взаимодействие между людьми, вещами, данными и процессами, объединенное в общую систему, цель которой – помочь пользователям принимать оптимальные решения. Кроме этого существует отдельное направление – промышленный интернет вещей («Industrial Internet of Things») – взаимосвязанные датчики, исполнительные устройства, промышленное оборудование и другие девайсы, объединенные в сеть совместно с промышленным программным обеспечением для управления производством. Эта взаимосвязь позволяет обмениваться данным между устройствами («M2M взаимодействие»), анализировать их и вносить изменения в производственный процесс для получения наибольшей производительности. Данная технология позволяет реализовать концепцию умного предприятия в рамках индустрии 4.0 [1].

Внедрение технологий промышленного интернета вещей в производство приводит к переходу от классической структуры АСУ ТП, представленной в виде пирамиды и состоящей из четырех уровней: 1) уровень планирования ресурсов (ERP); 2) уровень управления процессами (SCADA); 3) уровень контроля (ПЛК, HMI); 4) физический уровень (датчики, приводы); к структуре, в которой планирование ресурсов предприятия, управление процессами и контроль осуществляется в облачной платформе.

Основными элементами промышленного интернета вещей являются (рисунок 1):

1) облачные вычисления;

2) высокоскоростная сеть реального времени;

3) физический уровень (датчики, приводы).

Рисунок 1 Переход от традиционных АСУ ТП к IIOT

Применение данной технологии позволяет легко масштабировать существующие производственные участки, производство в целом, использовать данные, полученные с сенсоров оборудования для оценки его остаточного ресурса, а также для предотвращения нештатных ситуаций, приводящих к необратимым отказам оборудования.

Цель данного курсового проекта: создание программной части сервера устройства для облачной системы управления модулем транспортно-накопительной системы (складом) на основе облачной платформы «Yandex Cloud».

В ходе работы над проектом были поставлены следующие **задачи**:

1) проанализировать существующее ПО, управляющее складом и выявить его недостатки;
2) на основе анализа полученных данных сформулировать требования, предъявляемые к проектируемой программной части сервера устройства;
3) разработать функциональную модель перемещения паллет складом;
4) разработать алгоритмы основных функций программы;
5) разработать исходный код программной часть сервера устройства на языке C#.

1. Анализ существующей системы

В качестве анализируемой системы была выбрана ГПС производства фирмы «Denford», в частности один из модулей ее транспортно-накопительной системы - склад, расположенный в лабораторной аудитории 297 кафедры «Компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана. Структурная схема ГПС «Denford» приведена на рисунке 2.

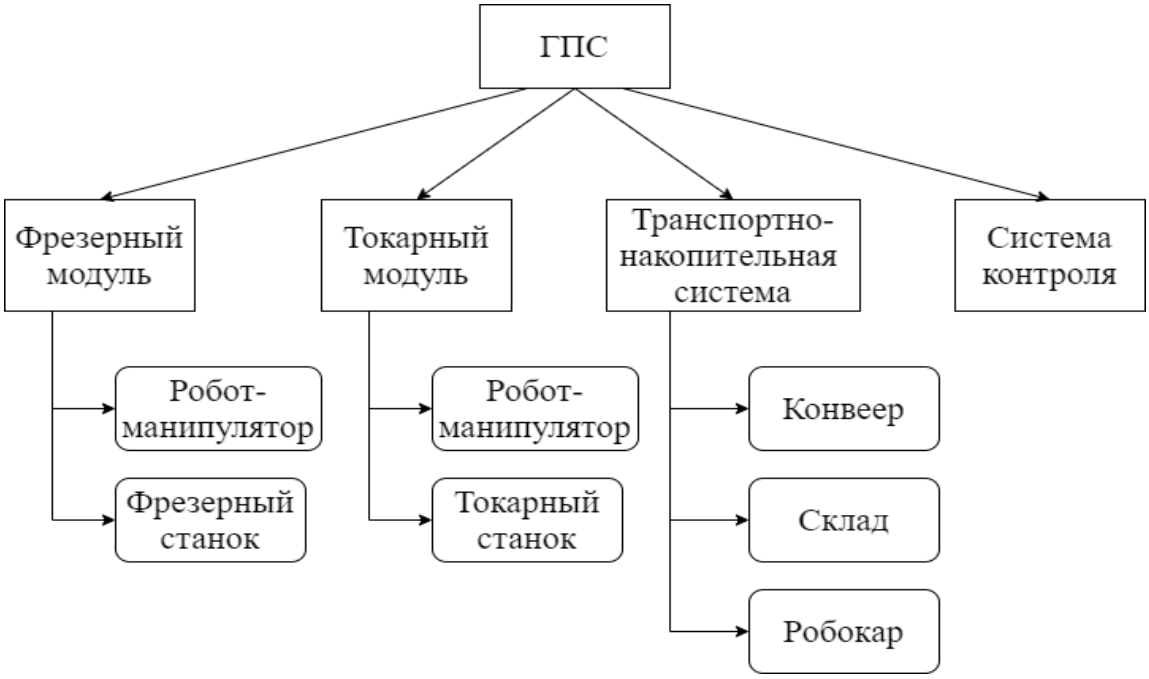


Рисунок 2 Структурная схема ГПС «Denford»

Склад представляет собой портальный манипулятор, управляемый ПЛК «NextMove-ESB-2» производства компании «ABB». Привод манипулятора электрический, рабочий элемент склада (штабелер) перемещается по трем направлениям декартовой системы координат с помощью трех сервоприводов «MicroFlex» производства компании «ABB», каждый из которых состоит из шагового двигателя, оптического энкодера, блока управления. Для предотвращения аварийных ситуаций на концах каждой из трех осей предусмотрены концевые выключатели. Структурная схема склада представлена на рисунке 3.

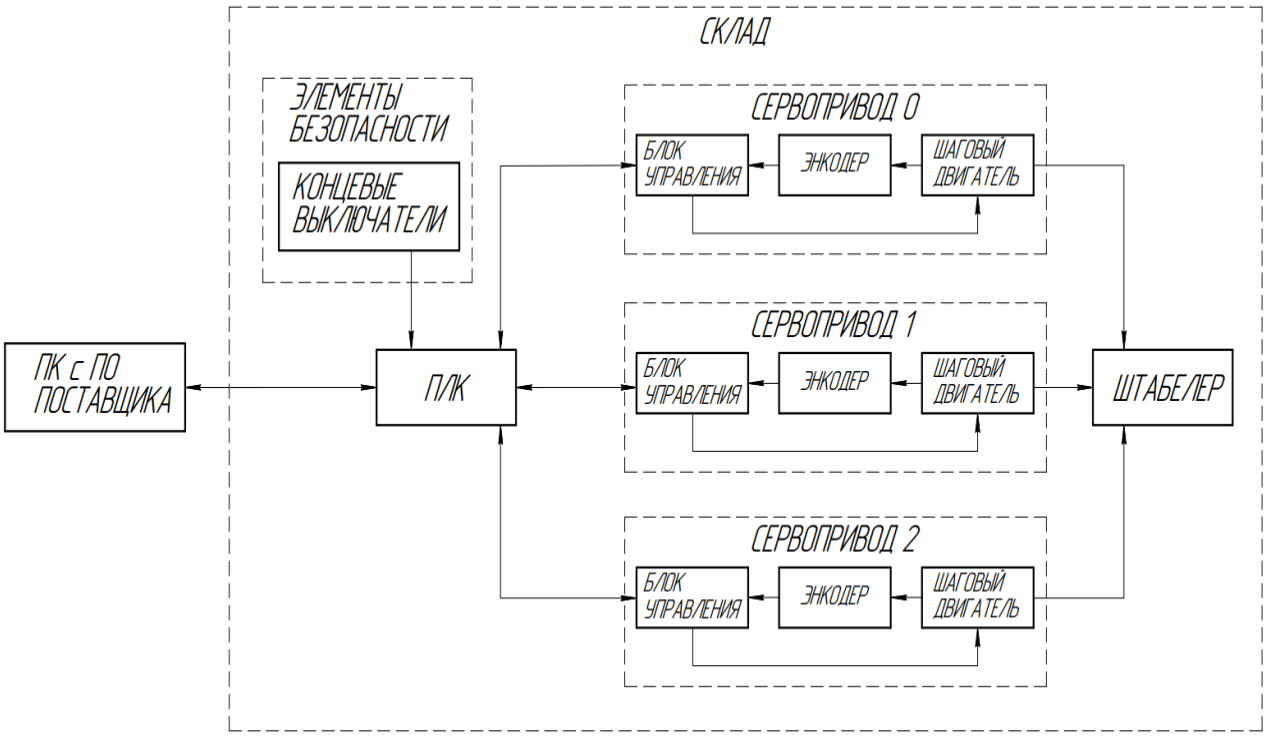


Рисунок 3 Структурная схема склада

В существующей системе управление склада осуществляется непосредственно оператором с помощью программного обеспечения производителя, установленного на ПК, подключенном к ПЛК через интерфейс USB. Данное программное решение имеет ряд существенных недостатков:

- отсутствие режима автономной работы;
- отсутствие возможности подключения к облачной платформе Yandex Cloud;
- закрытый исходный код, не позволяющий вносить изменения;
- ограниченный функционал (существует только один режим перемещения между «выученными» положениями).

Исходя из существующих недостатков ПО, предоставляемого производителем, можно сформулировать ряд требований к функционалу программной части сервер-устройства для управления складом:

- программа должна поддерживать как автономный режим работы устройства управления, так и управление оператором на месте;
- программа должна свободно взаимодействовать с облачной платформой Yandex Cloud;
- программа должна иметь открытый исходный код, позволяющий другим пользователям изменять его в зависимости от их целей;
- программа должна предоставлять несколько режимов работы: движение по заданным положениям, перемещение на фиксированное расстояние в одном из трех декартовых направлений;
- программа должна обладать интуитивно понятным интерфейсом.

2. Разработка функциональной модели перемещения паллет складом

В ходе работы над проектом по созданию программной части сервера устройства для управления складом на базе облачной платформы «Yandex Cloud» была разработана функциональная модель программы (рисунок 4).

Рисунок 4 Блок А-0

Функциональная модель системы отражает этапы транспортировки паллеты внутри транспортно-накопительного модуля ГПС «Denford» в автоматическом режиме, начиная с получения управляющей команды сервером устройства от облачной платформы и заканчивая непосредственным перемещением паллеты в требуемое положение (рисунок 5).

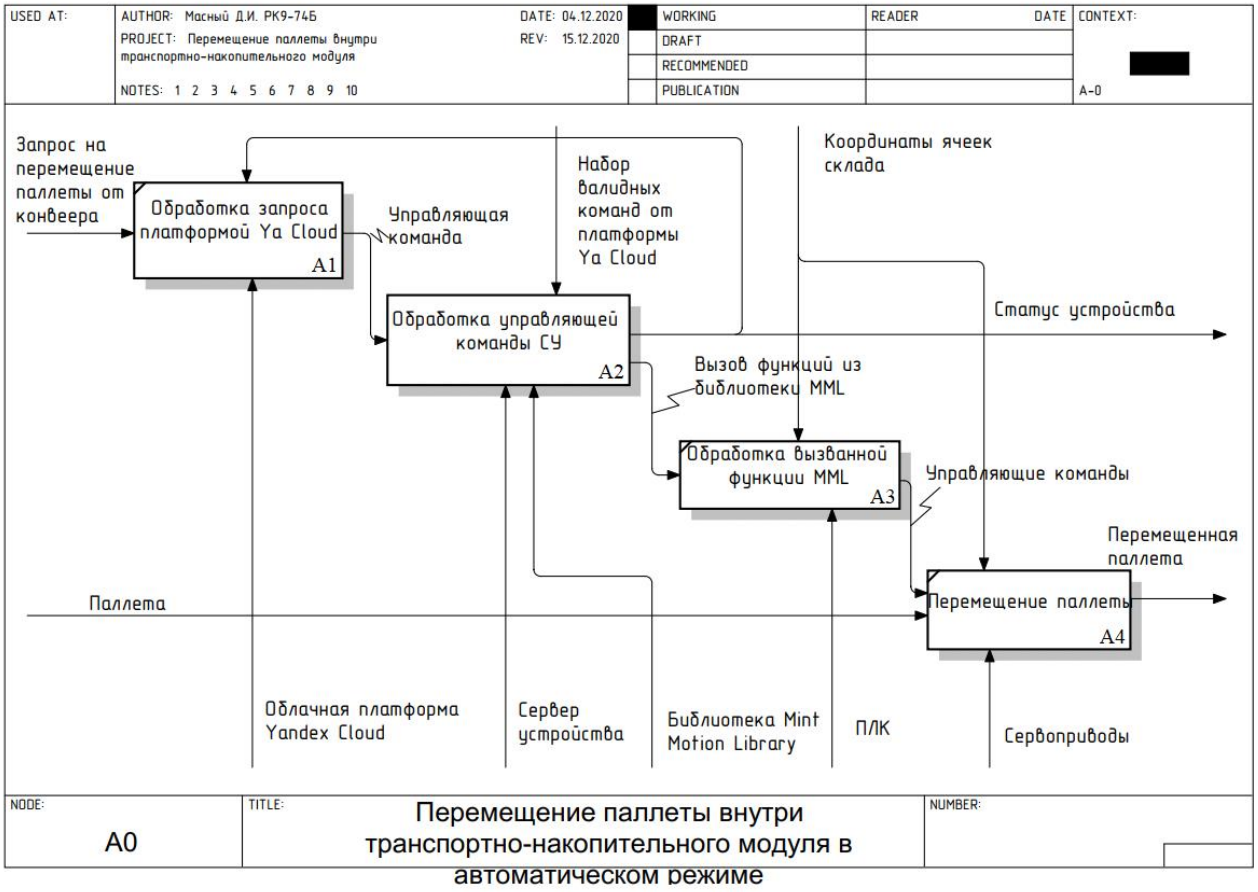


Рисунок 5 Блок A0

Процесс транспортировки начинается с обработки облачной платформой запроса на перемещение паллеты от конвейера, далее «Yandex Cloud» отправляет управляющую команду серверу устройства. Сервер устройства обрабатывает полученную команду. Если получена валидная команда от облачной платформы, то сервер устройства запускает необходимую подпрограмму и отправляет статус «processing» в облако. В случае получения невалидных данных, отправляется статус «not valid». Запущенная подпрограмма вызывает необходимые функции библиотеки «Mint Motion Library» с параметрами, переданными из облака. Используя элемент управления «ActiveX» и «MIL-сервер», север устройства, подключенное к ПЛК через интерфейс USB, передает информацию о том, какие функции были вызваны, контроллеру. Системное ПО ПЛК обрабатывает эту информацию и отправляет соответствующие управляющие команды сервоприводам, которые, в свою очередь, начинают перемещать паллету. По

завершении перемещения паллеты сервер устройства отправляет статус «ready» в облако.

Этап обработки управляющей команды сервер-устройством был разобран более подробно. Функциональная схема представлена на рисунке 6.

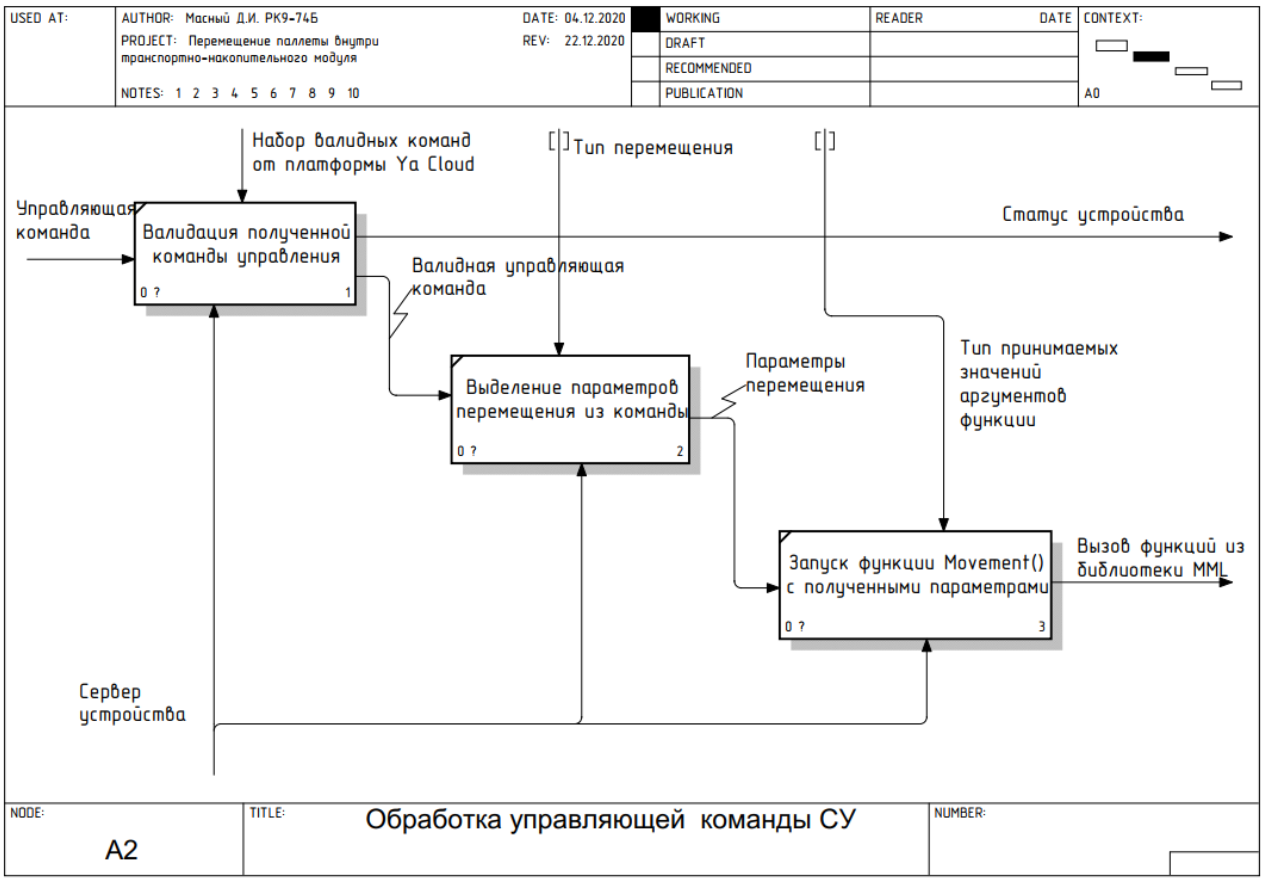


Рисунок 6 Блок A2

Процесс обработки команды от облачной платформы сервером устройства начинается с валидации полученных данных. Данные сверяются с набором допустимых команд, разработанных в процессе создания проекта. Если были получены невалидные данные, сервер устройства отправляет в облако статус «not valid». В случае прохождения проверки, из полученных данных выделяются параметры, необходимые для перемещения (начальное и конечное положения). Далее вызывается функция перемещения с указанными параметрами, внутри которой происходит вызов функций библиотеки «MML».

3. Разработка концептуальной модели

В ходе работы над проектом была разработана концептуальная модель взаимодействия между облачной платформой «Yandex Cloud», сервером устройства и ПЛК складского модуля (рисунок 7). Данная модель состоит из следующих сущностей:

- сервер-устройство;
- программа;
- платформа «Yandex Cloud»;
- команда;
- состояние;
- ПЛК;
- сервопривод.

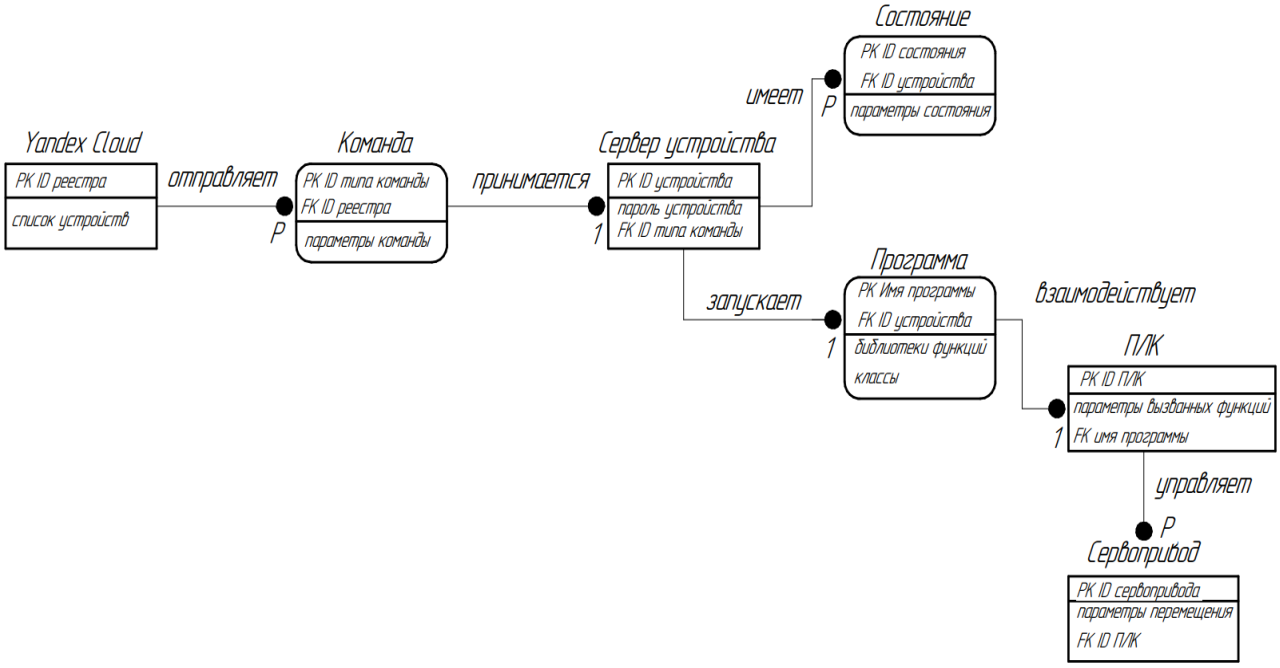


Рисунок 7 Концептуальная модель взаимодействия

4. Взаимодействие склада с облачной платформой «Yandex Cloud»

Так как у используемого оборудования (в частности ПЛК) отсутствует возможность подключения к интернету, взаимодействие промышленного оборудования с облачной платформой «Yandex Cloud» осуществляется через сервер-устройство. В качестве сервера устройства выступает ПК с архитектурой x86 и установленной ОС «Windows 10», однако может быть использовано любое другое устройство, поддерживающее установку полноценной операционной системы «Windows», например, одноплатный компьютер «Rock Pi X» [7]. Использование именно операционной системы «Windows» обусловлено тем, что для взаимодействия между сервером устройства и ПЛК склада используется элемент управления «ActiveX». Данная технология была разработана компанией «Microsoft» и поддерживается только ОС «Windows». Обмен данными между облачной платформой «Yandex Cloud» и сервером устройства осуществляется через интерфейс Wi-Fi по протоколу MQTT. Передача данных между сервером устройства и ПЛК осуществляется через интерфейс USB1.1 с использованием протокола USB-шины (рисунок 8).

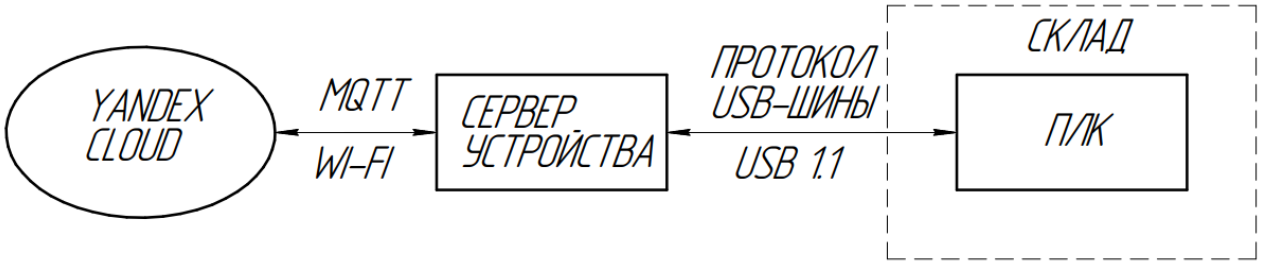


Рисунок 8 Схема взаимодействия устройств

4.1 Взаимодействие сервера устройства с облачной платформой «Yandex Cloud»

Взаимодействие сервера устройства, в роли которого выступает ПК с ОС «Windows 10», с облачной платформой «Yandex Cloud» осуществляется с помощью интерфейса беспроводной связи Wi-Fi по протоколу MQTT.

Протокол MQTT (Message Queue Telemetry Transport) – это легкий, компактный и открытый протокол обмена данными, созданный для передачи информации на удалённых локациях, где объем передаваемых данных невелик и есть ограничения по пропускной способности канала [6]. К основным преимуществам протокола MQTT можно отнести следующие:

- компактные передаваемые сообщения;
- работа в условиях нестабильного соединения;
- легкая интеграция новых устройств;
- поддержка нескольких уровней качества обслуживания.

Протокол MQTT работает на прикладном уровне поверх протокола транспортного уровня TCP и протокола сетевого уровня IP (рисунок 9) [6].

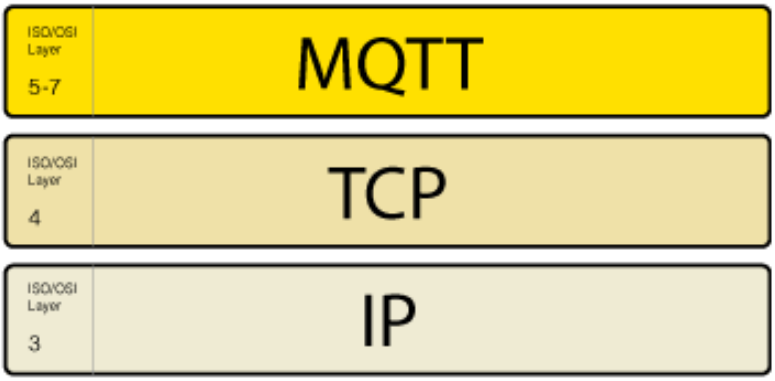


Рисунок 9 Уровни протоколов передачи данных

Обмен сообщениями в протоколе MQTT осуществляется между клиентом («client»), который может быть издателем («publisher») или подписчиком («subscriber») сообщений, и брокером («broker») сообщений. Платформа

«Yandex Cloud» предоставляет доступ к собственному MQTT-брокеру по адресу mqtt.cloud.yandex.net, используя 8883 порт для подключения [9].

Издатель отправляет данные на MQTT-брокер, указывая в сообщении определенную тему, топик («topic»). Подписчики данного топика могут получать разные данные от множества издателей, также один издатель может публиковать сообщения в множество топиков.

Устройства MQTT используют определенные типы сообщений для взаимодействия с брокером, ниже представлены основные:

- «connect» – установить соединение с брокером;
- «disconnect» – разорвать соединение с брокером;
- «publish» – опубликовать данные в топик на брокере;
- «subscribe» – подписаться на топик на брокере;
- «unsubscribe» – отписаться от топика.

Для наглядного описания взаимодействия между издателем, брокером и клиентом была составлена UML-диаграмма последовательности (рисунок 10).

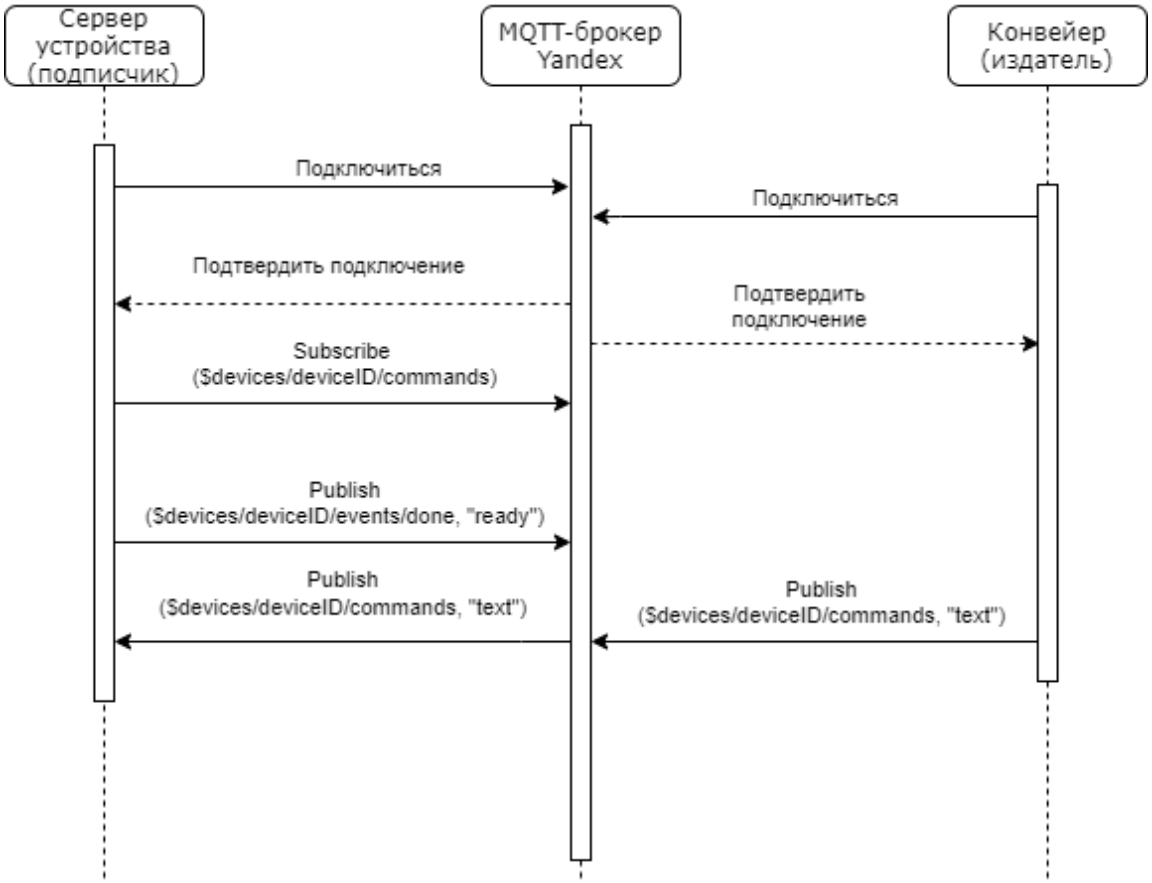


Рисунок 10 UML-диаграмма последовательности взаимодействия сервера устройства с облаком

Прежде всего создается TCP/IP соединение между сервером устройства и облачной платформой «Yandex Cloud». После чего сервер устройства подключается к MQTT-брокеру, получает подтверждение подключения и подписывается на определенный топик брокера «Yandex».

В данном проекте для подписки клиента на определенный топик используется следующий формат сообщения: «$devices/deviceID/commands», где «deviceID» – идентификационный номер устройства, присвоенный ему платформой «Yandex Cloud» при регистрации в облачном сервисе, «commands» – название топика, в который MQTT-брокер публикует команды [9]. После подписки устройства на топик «commands» для получения управляющих команд устройство публикует свой статус «ready» в сабтопик «/events/done», если оно готово к приему управляющих команд, и статус «not ready», если не готово.

В роли издателя может выступать любое устройство, входящее в состав одного из модулей ГПС, но в данном проекте подразумевается, что этим устройством является конвейер, который после подключения к MQTT-брокеру, подтверждает подключение, публикует сообщение о том, что необходимо забрать паллету в топик «/commands», на который подписан сервер устройства, управляющее складом. После публикации сообщения издателем в определенный топик все подписчики данного топика получают эту информацию.

Для защиты передаваемых данных между устройствами используется криптографический протокол «TSL 1.2». Данный протокол предусматривает аутентификацию устройств в сети как с использованием X509 сертификатов, так и с использованием логина и пароля [9]. В курсовом проекте аутентификация устройства происходит по логину и паролю, где в качестве логина выступает ID устройства, присваиваемое ему облачной платформой «Yandex» при его регистрации, пароль же задается пользователем. Использование шифрования позволяет минимизировать риск кражи передаваемых данных третьими лицами, потому что доступ к логину и паролю каждого устройства ограничен.

4.2 Формат сообщений между облачной платформой и сервером устройства

Для обмена данными между сервером устройства, управляющим складом, и облачной платформой «Yandex Cloud» было разработано 2 типа сообщений.

Первый тип сообщений – сообщения, передающие статус складского модуля:

1) «ready» – сервер устройства готов к получению сообщений от облачной платформы;
2) «not ready» – сервер устройства не готов к получению сообщений от облачной платформы;
3) «processing» – склад выполняет полученную команду.

Второй тип сообщений – управляющие команды, которые передает облачная платформа серверу устройства:

1) «from conveyor to column X row Y» – команда, инициирующая перемещение паллеты с конвейера в нужную ячейку с координатами (X,Y), где X – номер столбца ячейки, Y – номер ряда, могут принимать значения от 1 до 5 каждая;
2) «to conveyor from column X row Y» – команда, инициирующая перемещение паллеты из ячейки склада с координатами (X,Y), где X – номер столбца ячейки, Y – номер ряда на конвейер;
3) «empty column X row Y to column M row N pillar Z» – команда, инициирующая перемещение паллеты внутри складского модуля между ячейками, где (X,Y) – координаты начальной ячейки, (M,N) – координаты конечной ячейки, они могут принимать значения от 1 до 5 каждая. Z – параметр выбора стеллажа, левый или правый, может принимать значения («left» либо «right»).

4.3 Взаимодействие сервера устройства с ПЛК склада

Взаимодействие сервера устройства, в роли которого выступает ПК с ОС «Windows 10», с ПЛК, управляющим сервоприводами склада, осуществляется с помощью интерфейса USB1.1 с использованием протокола USB-шины.

Протокол шины USB обеспечивает обмен данными между хостом (сервером устройства) и устройством (ПЛК склада). На протокольном уровне решаются такие задачи, как обеспечение достоверности и надежности передачи. Все данные на шине USB передаются посредством транзакций, в каждой транзакции возможен обмен только между хостом и адресуемым устройством. Все транзакции (обмены) с устройствами USB состоят из двух служебных пакетов («Token packet», «Status packet») и одного пакета данных («Data packet»). Схема работы протокола USB-шины представлена на рисунке 11.

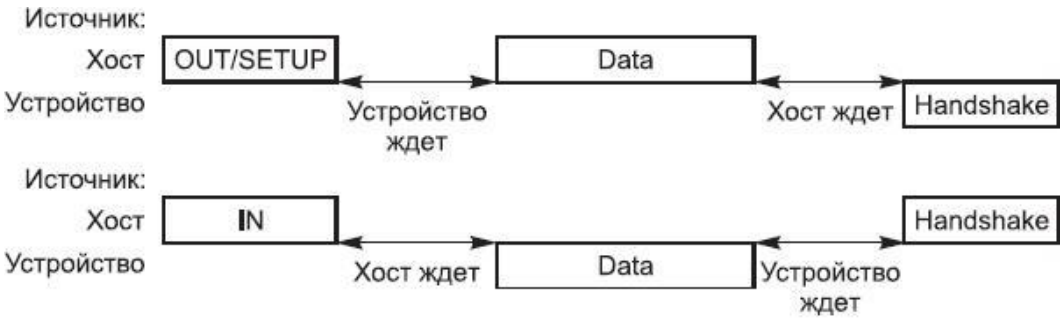


Рисунок 11 Схема передачи данных по протоколу USB

Каждая транзакция планируется и начинается по инициативе хоста (сервера устройства), который посылает пакет-маркер транзакции («Token packet»). Маркер транзакции описывает тип и направление передачи, адрес выбранного устройства USB. Далее адресуемое маркером устройство распознает свой адрес и готовится к обмену. Источник данных, определенный маркером, передает пакет данных. И в завершении транзакции следует пакет статуса («Status Packet»), который либо подтверждает успешное принятие данных, либо указывает, что данные переданы неверно [11].

4.4 Использование библиотеки «Mint Motion Library»

Разработанная компанией «ABB» библиотека «Mint Motion Library» позволяет создавать приложения на базе ПК, взаимодействующие с ПЛК с помощью элемента управления «Mint ActiveX» и «MIL-сервера».

Элементы управления «ActiveX» представляют собой объекты, которые встраиваются в программу, для того чтобы предоставить пользователю возможность управлять промышленным оборудованием с помощью приложений на базе ПК, взаимодействующих с ПЛК.

Преимуществом элементов управления «ActiveX» является то, что они могут использоваться в приложениях, написанных на многих языках программирования, включая: C#, C++, Delphi, Visual Basic.

Элемент управления «Mint ActiveX» предоставляет API (программный интерфейс приложения) для ПЛК и является компонентом пользовательского интерфейса, добавляемым в окна и формы пользовательских программ. «MIL-сервер» обрабатывает связь элемента управления «Mint ActiveX» с контроллером. «MIL-сервер» начинает свою работу, когда создается первый элемент управления «Mint ActiveX», и завершает свою работу, когда элемент управления уничтожается [5]. Схема взаимодействия сервера устройства с ПЛК представлена на рисунке 12.

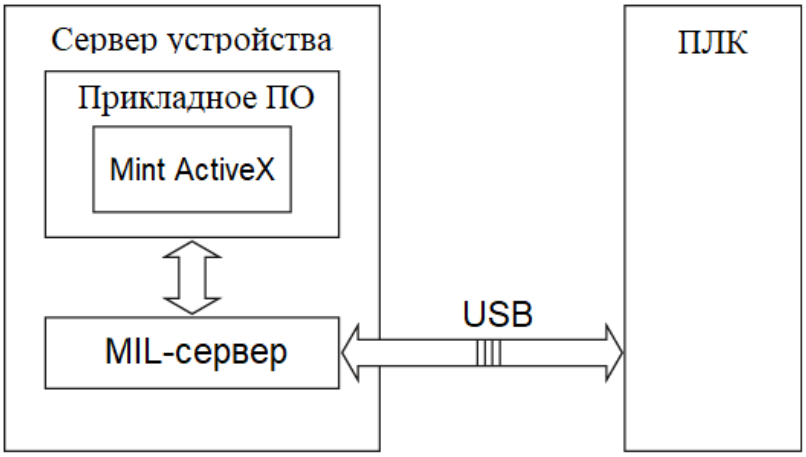


Рисунок 12 Структурная схема взаимодействия сервера устройства с ПЛК

Для наглядного описания взаимодействия между сервером устройства и ПЛК была составлена UML-диаграмма последовательности (рисунок 13).

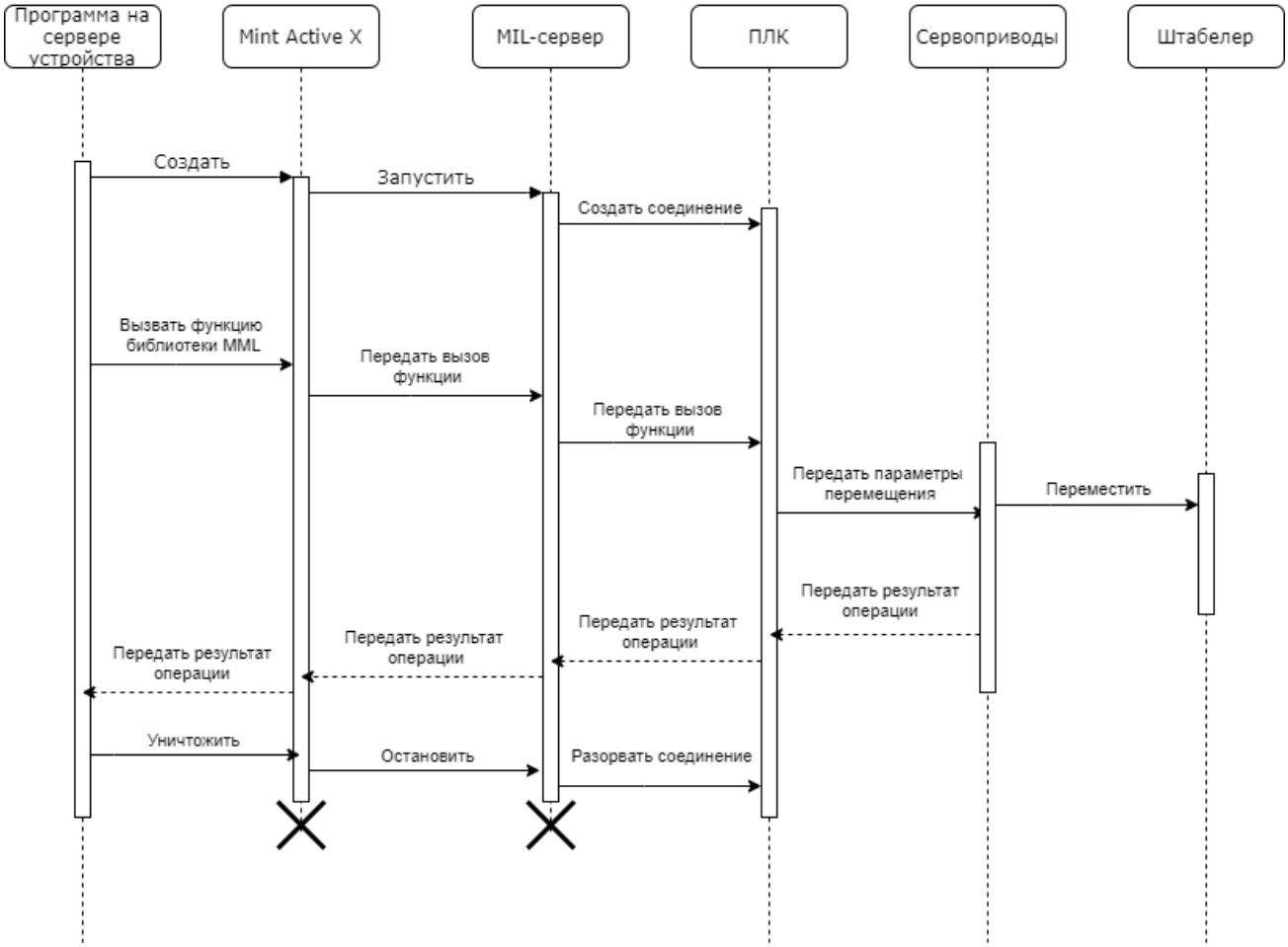


Рисунок 13 UML-диаграмма последовательности взаимодействия сервера устройства с ПЛК

После создания элемента управления «Mint ActiveX» «MIL-сервер» начинает свою работу и устанавливает соединение с ПЛК через интерфейс USB1.1. Когда прикладное ПО сервера устройства вызывает функцию библиотеки «Mint Motion Library», элемент управления «ActiveX» взаимодействует через «MIL-сервер» с системным ПО ПЛК и передает информацию о том, какая функция библиотеки должна быть вызвана и с какими параметрами. Системное ПО ПЛК обрабатывает вызванную функцию и выполняет требуемые команды или возвращает серверу устройства сообщение об ошибке [5].

Стоит отметить, что написание приложений на базе ПК для взаимодействия с исполнительными устройствами склада оправдано только тогда, когда необходимо производить обработку больших вычислений или, как в данном проекте, при подключении оборудования к облачной платформе.

5. Разработка программной части сервера устройства

Прежде чем приступать к написанию программной части сервера устройства, необходимо разработать алгоритмы работы основных функций программы. К числу основных функций программы можно отнести следующие:

1) подключение сервера устройства к облачной платформе и подписка на нужные топики;
2) валидация полученных команд от облачной платформы;
3) перемещение штабелера.

5.1 Проектирование алгоритма подключения сервера устройства к облачной платформе «Yandex Cloud»

Вся логика взаимодействия сервера устройства, облачной платформы, параметры подключения описаны в классе YaClient (приложение А). Кроме этого в программе реализован событийный подход, то есть выполнение определенного блока кода определяется событием [2]. В данном случае событием является нажатие пользователям кнопок экранной формы. Алгоритм функции подключения представлен на рисунке 14.

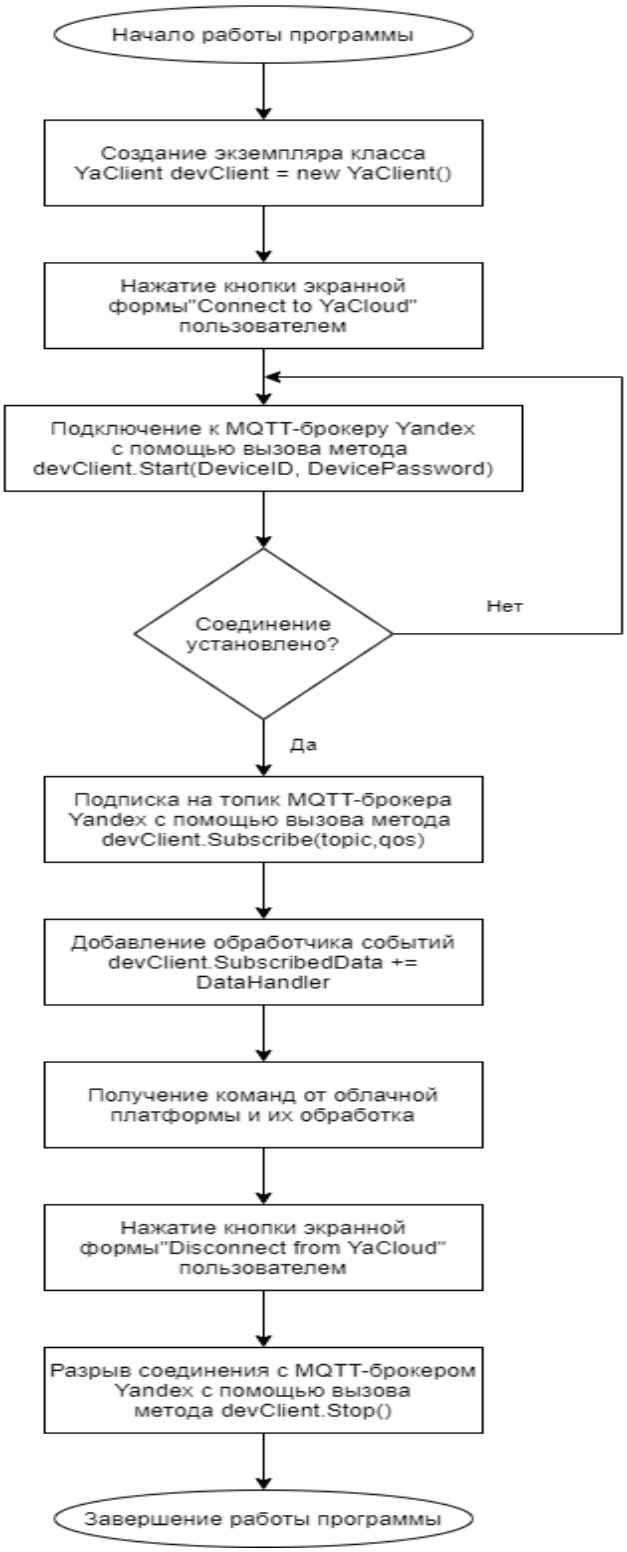


Рисунок 14 Алгоритм подключения сервер-устройства к облачной платформ

5.2 Проектирование алгоритма функции валидации полученных команд от облачной платформы «Yandex Cloud»

После подключения сервера устройства к MQTT-брокеру и подписки на определенный топик, устройство готово принимать команды от облачной платформы. Как и в предыдущем случае, используется парадигма событийного программирования, только сейчас событием является получение данных из топика (рисунок 15). Исходный код представлен в приложении Б.

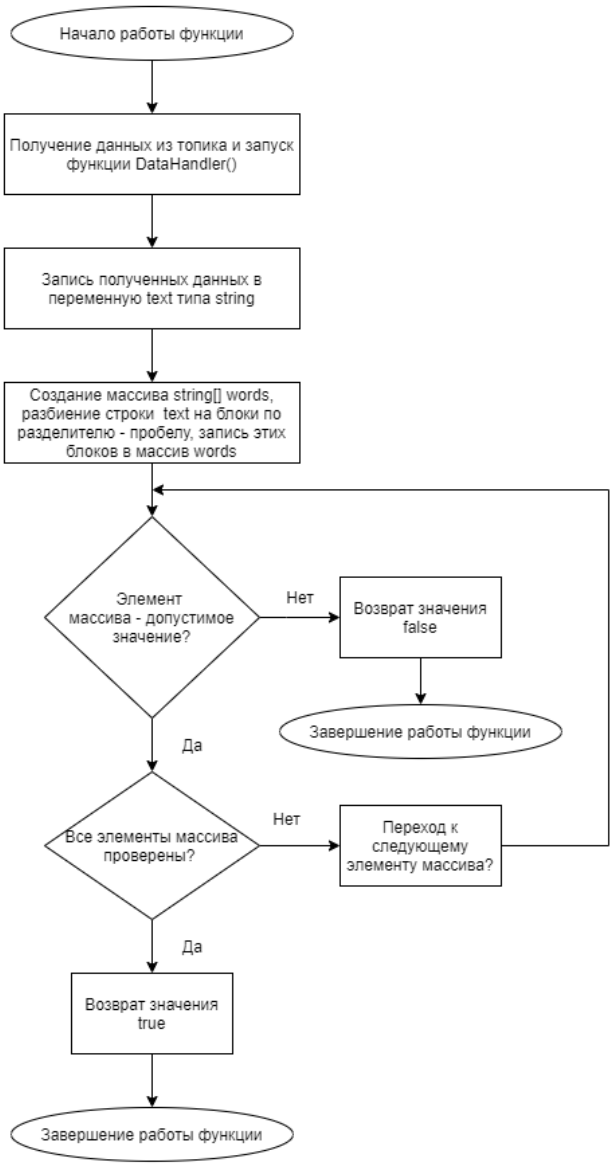


Рисунок 15 Алгоритм валидации полученных данных от облака

5.3 Проектирование алгоритма функции перемещения штабелера

Сначала необходимо определить координаты всех ячеек склада, а также координаты конвейера, чтобы штабелер мог забирать паллеты оттуда. Стоит отметить, что точкой отсчета в используемой системе координат является крайнее правое положение штабелера, при котором нажаты концевые выключатели на каждой из его осей. Кроме этого координаты получены не в миллиметрах, а шагах двигателей, отвечающих за перемещение штабелера по осям X, Y, Z (приложение В). Это сделано для того, чтобы без дополнительных вычислений сверять реальное положение штабелера с заданным, поскольку библиотека «Mint Motion Library» предоставляет набор функций, возвращающих текущие показания энкодеров в шагах. По умолчанию 200 шагов двигателя равны 1 миллиметру.

В используемых сервоприводах применяются инкрементальные энкодеры, которые после отключения питания обнуляют свое положение в памяти сервоприводов, но штабелер физически остается на месте, поэтому сразу после включения складского модуля его необходимо предварительно вывести в нулевое положение [4]. Перемещение разрешено, если штабелер был выведен в нулевое положение и команда от облака прошла валидацию. Алгоритм функции перемещения представлен на рисунке 16. Исходный код функции представлен в приложении Б.

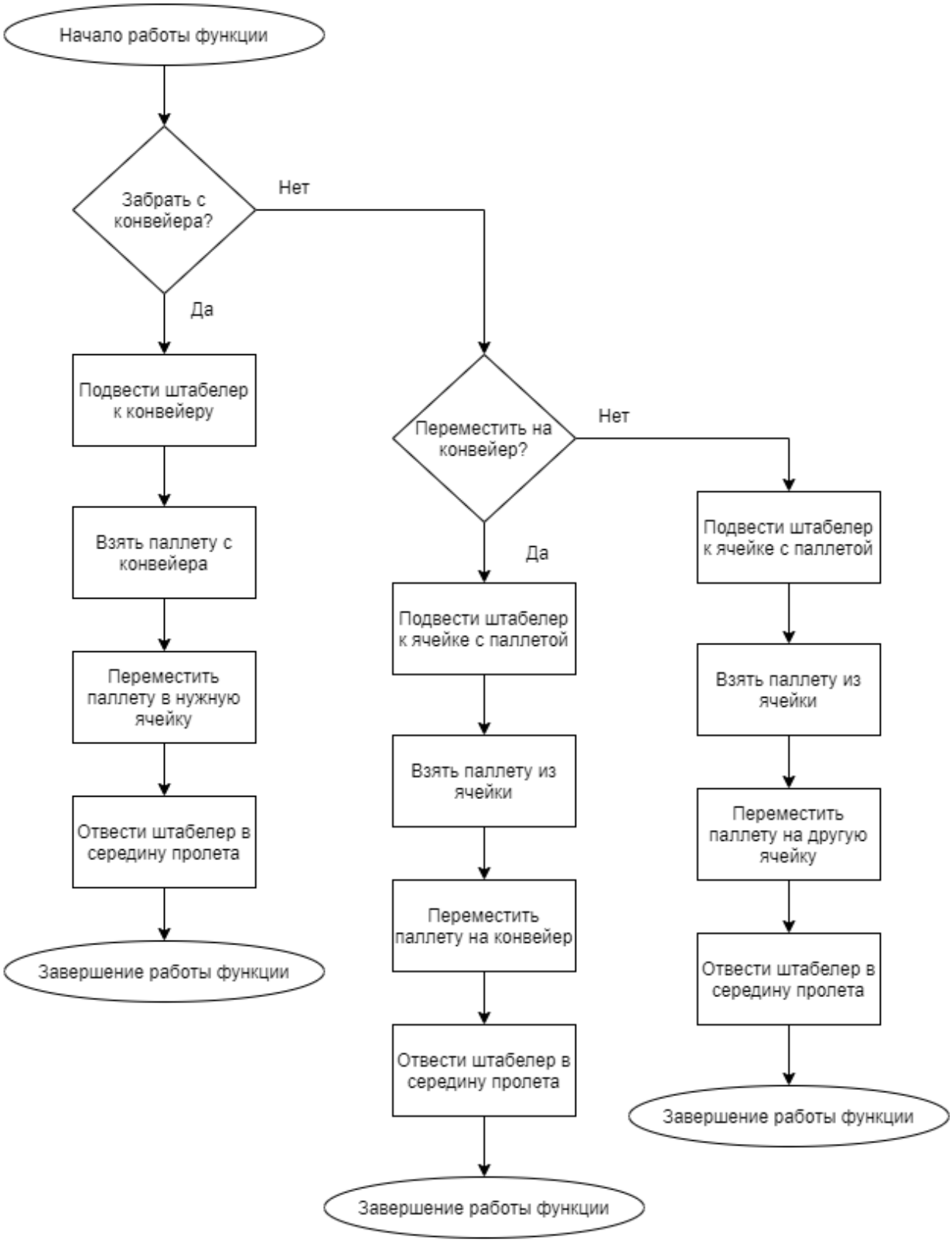


Рисунок 16 Алгоритм функции перемещения штабелера

5.4 Разработка графического интерфейса программы

На основе требования к интерфейсу программной части сервера устройства была разработана экранная форма (рисунок 17). Данная форма поддерживает два сценария работы в зависимости от задач пользователя: «автономный» и «режим управления оператором».

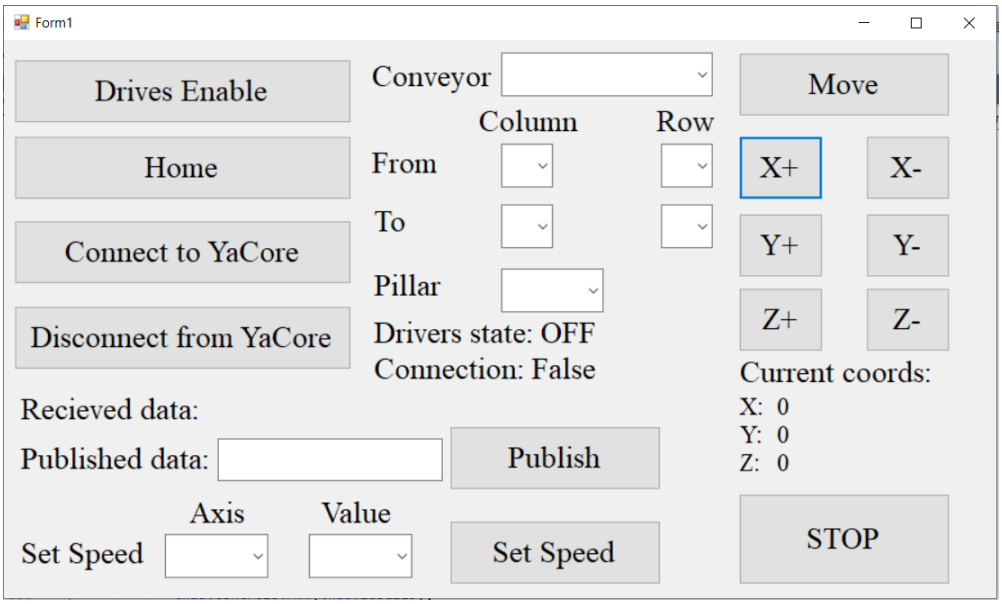


Рисунок 17 Экранная форма разработанной программной части

При реализации сценария «автономный» оператор ГПС выполняет ряд действий:

1. запускает программную часть сервера устройства;
2. включает сервоприводы склада путем нажатия на кнопку «Drives Enable»;
3. вводит штабелер склада в нулевое положение путем нажатия кнопки «Home»;
4. подключает сервер устройство к облачной платформе «Yandex Cloud» путем нажатия кнопки «Connect to YaCloud»;
5. по завершении рабочей смены оператор отключает сервер устройство от «Yandex Cloud» путем нажатия кнопки «Disconnect from YaCloud»;
6. закрывает экранную форму.

При реализации сценария «режим управления оператором» оператор выполняет следующие действий:

1. запускает программную часть сервера устройства;
2. включает сервоприводы склада путем нажатия кнопки «Drives Enable»;
3. выводит штабелер склада в нулевое положение путем нажатия кнопки «Home»;
4. если необходимо переместить паллету с какой-либо конкретной ячейки, оператор вводит параметры в соответствующие «combobox»: «Conveyor» – может принимать параметры «From» (снять с конвейера), «To» (положить на конвейер), «Empty» (конвейер не задействован в перемещении); «From» – вводятся координаты начальной ячейки; «To» – вводятся координаты конечной ячейки; «Pillar» – вводится значение либо «left» (левый стеллаж), либо «right» (правый стеллаж);
5. путем нажатия кнопки «Move» запускается перемещение паллеты в соответствии с параметрами, указанными в «combobox»;
6. если необходимо переместить штабелер склада в какое-либо произвольное положение, то используются кнопки «X+» и «X-» (для перемещения штабелера по оси X в положительном и отрицательном направлении на 1 мм соответственно), «Y+» и «Y-» (для перемещения штабелера по оси Y в положительном и отрицательном направлении на 1 мм соответственно), «Z+» и «Z-» (для перемещения штабелера по оси Z в положительном и отрицательном направлении на 1 мм соответственно);
7. по завершении рабочей смены оператор закрывает экранную форму.

ЗАКЛЮЧЕНИЕ

В ходе работы над проектом были получены следующие результаты:

1) В настоящее время внедряются кибер-физические системы и технологии промышленного интернета вещей в производство. Это приводит к переходу от традиционных АСУ ТП к «облачным», в которых планирование ресурсов предприятия, управление процессами и контроль осуществляется в облачной платформе.
2) Анализ существующего программного обеспечения, управляющего складским модулем, показал, что данное ПО имеет ряд недостатков, основным из которых является отсутствие подключения к облачной платформе «Yandex Cloud».
3) Были сформированы требования к разрабатываемой программе, важнейшее из которых – возможность автономной работы складского модуля при подключении к облачной платформе «Yandex Cloud».
4) Были разработаны: функциональная модель перемещения паллет складом, концептуальная модель взаимодействия облачной платформы, сервера устройства и ПЛК складского модуля.
5) Были разработаны алгоритмы: функции подключения сервера устройства к облачной платформе, функции валидации полученных команд от платформы «Yandex Cloud», функции перемещения штабелера склада.
6) Был разработан исходный код программной части сервера устройства на языке C#.

Разработанная программная часть сервера устройства совместно с облачной платформой «Yandex Cloud» реализуют концепцию промышленного интернета вещей в рамках рассматриваемого складского модуля ГПС «Denford».

Программа позволяет оператору взаимодействовать со складом через интуитивно понятную экранную форму по одному из двух сценариев: 1) «автономный» - команды на перемещение паллет передаются серверу устройства

по беспроводной сети через протокол MQTT от облачной платформы «Yandex Cloud»; 2) «режим управления оператором» - управление складом осуществляется непосредственно оператором через сервер устройства путем заполнения соответствующих полей экранной формы. Выбор сценария осуществляет оператор ГПС в зависимости от его целей и задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Buyya, R. Internet of Things Principles and Paradigms/ R. Buyya, A.V. Dastjerdi – Singapore: Elsevier, 2016. – 354 p.
2. Kudchikar, S. Delegates and events in C# [Электронный ресурс] // Codewithshadnam [сайт]. [2019]. URL: https://codewithshadman.com/delegates-and-events-in-csharp/ (дата обращения 28.11.2020)
3. Lee, E. A. Introduction to Embedded Systems – A Cyber-Physical Systems Approach / E. A. Lee, S. A. Seshia – Berkley: UC Berkley, 2011. – 519 p.
4. MicroFlex Servo Control: Installation Manual – Bristol: Baldor UK Ltd, 2010. – 124 p.
5. Mint Help version HF0004A08 – ABB, 2017. – 217 p.
6. Puler, T. Hands-On Internet of Things with MQTT – Birmingham, Packt, 2019. – 350 p.
7. Rock Pi X getting started [Электронный ресурс] // Wiki.Radxa [сайт]. [2020] URL: https://wiki.radxa.com/RockpiX/getting_started#get_start_specs (дата обращения 05.11.2020)
8. Овсянников, М.В. Организация индустриального интернета на основе SAP HANA Cloud Platform: учебное пособие/М.В. Овсянников, С.А. Буханов, С.А. Подкопаев. – М.: издательство МГТУ им. Н. Э. Баумана, 2019. – 195 с.
9. Пошаговые инструкции Yandex.Cloud – Документация [Электронный ресурс] // Cloud.Yandex [сайт]. [2020] URL: https://cloud.yandex.ru/docs/iot-core/operations/publish (дата обращения 20.11.2020)
10. Протокол USB-шины [Электронный ресурс] // Perscom [сайт]. [2020]. URL: http://perscom.ru/usb (дата обращения 17.09.2020)
11. Соколов, Д. Индустрия 4.0: Big Data, цифровизация и рост экономики [Электронный ресурс] // Habr [сайт]. [2020]. URL: https://habr.com/ru/post/507822/ (дата обращения 12.10.2020)

Приложение А

1	Левый стеллаж, нумерация от входа			
2	№ ячейки	X(0)	Y(1)	Z(2)
3	Безопасное положение	0	-88181	0
4	Взять с конвеера	-495990	-88181	-131987
5	Поставить на конвеер	-495990	-88181	-144987
6	1	-148989	-39173	-23987
7	2	-200739	-39173	-23987
8	3	-252489	-39173	-23987
9	4	-304239	-39173	-23987
10	5	-355989	-39173	-23987
11	6	-148989	-39173	-62984
12	7	-200739	-39173	-62984
13	8	-252489	-39173	-62984
14	9	-304239	-39173	-62984
15	10	-355989	-39173	-62984
16	11	-148989	-39173	-125968
17	12	-200739	-39173	-125968
18	13	-252489	-39173	-125968
19	14	-304239	-39173	-125968
20	15	-355989	-39173	-125968
21	16	-148989	-39173	-188952
22	17	-200739	-39173	-188952
23	18	-252489	-39173	-188952
24	19	-304239	-39173	-188952
25	20	-355989	-39173	-188952
26	21	-148989	-39173	-251936
27	22	-200739	-39173	-251936
28	23	-252489	-39173	-251936
29	24	-304239	-39173	-251936
30	25	-355989	-39173	-251936

1	Правый стеллаж, нумерация от входа			
2	№ ячейки	X(0)	Y(1)	Z(2)
3	Безопасное положение	0	-88181	0
4	Взять с конвеера	-495990	-88181	-131987
5	Поставить на конвеер	-495990	-88181	-144987
6	26	-148989	-136189	-23987
7	27	-200739	-136189	-23987
8	28	-252489	-136189	-23987
9	29	-304239	-136189	-23987
10	30	-355989	-136189	-23987
11	31	-148989	-136189	-62984
12	32	-200739	-136189	-62984
13	33	-252489	-136189	-62984
14	34	-304239	-136189	-62984
15	35	-355989	-136189	-62984
16	36	-148989	-136189	-125968
17	37	-200739	-136189	-125968
18	38	-252489	-136189	-125968
19	39	-304239	-136189	-125968
20	40	-355989	-136189	-125968
21	41	-148989	-136189	-188952
22	42	-200739	-136189	-188952
23	43	-252489	-136189	-188952
24	44	-304239	-136189	-188952
25	45	-355989	-136189	-188952
26	46	-148989	-136189	-251936
27	47	-200739	-136189	-251936
28	48	-252489	-136189	-251936
29	49	-304239	-136189	-251936
30	50	-355989	-136189	-251936

Приложение Б

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net.Security;
using System.Security.Authentication;
using System.Security.Cryptography.X509Certificates;
using System.Threading;
using System.Threading.Tasks;
using MQTTnet;
using MQTTnet.Client;
using MQTTnet.Client.Connecting;
using MQTTnet.Client.Disconnecting;
using MQTTnet.Client.Options;

namespace YaClass
{
    public enum EntityType
    {
        Registry = 0,
        Device = 1
    }

    public enum TopicType
    {
        Events = 0,
        Commands = 1
    }

    public class YaClient
    {
        public const string MqttServer = "mqtt.cloud.yandex.net";
        public const int MqttPort = 8883;

        private static X509Certificate2 rootCrt = new
X509Certificate2("C:/Users/Zver/rootCA.crt");

        public static string TopicName(string entityId, EntityType
entity, TopicType topic)
        {
            string result = (entity == EntityType.Registry) ?
"$registries/" : "$devices/";
            result += entityId;
            result += (topic == TopicType.Events) ? "/events" :
"/commands";
            return result;
        }

        public delegate void OnSubscribedData(string topic, byte[]
payload);
        public event OnSubscribedData SubscribedData;
```

```
        private IMqttClient mqttClient = null;
        private ManualResetEvent oCloseEvent = new
ManualResetEvent(false);
        private ManualResetEvent oConnectedEvent = new
ManualResetEvent(false);



        public void Start(string id, string password)
        {
            //setup connection options
            MqttClientOptionsBuilderTlsParameters tlsOptions = new
MqttClientOptionsBuilderTlsParameters
            {
                SslProtocol = SslProtocols.Tls12,
                UseTls = true
            };
            tlsOptions.CertificateValidationCallback +=
CertificateValidationCallback;

            // Create TCP based options using the builder.
            var options = new MqttClientOptionsBuilder()
                .WithClientId($"Test_C#_Client_Dmasny_PC")
                .WithTcpServer(MqttServer, MqttPort)
                .WithTls(tlsOptions)
                .WithCleanSession()
                .WithCredentials(id, password)
                .Build();
            var factory = new MqttFactory();
            mqttClient = factory.CreateMqttClient();

mqttClient.UseApplicationMessageReceivedHandler(DataHandler);
            mqttClient.UseConnectedHandler(ConnectedHandler);

mqttClient.UseDisconnectedHandler(DisconnectedHandler);
            mqttClient.ConnectAsync(options,
CancellationToken.None);
        }
        public void Stop()
        {
            oCloseEvent.Set();
            mqttClient.DisconnectAsync();
        }
        public bool WaitConnected()
        {
            WaitHandle[] waites = { oCloseEvent, oConnectedEvent
};
            return WaitHandle.WaitAny(waites) == 1;
        }
        public Task Subscribe(string topic,
MQTTnet.Protocol.MqttQualityOfServiceLevel qos)
        {
```

```
            return mqttClient.SubscribeAsync(topic, qos);
        }

        public Task Publish(string topic, string payload,
MQTTnet.Protocol.MqttQualityOfServiceLevel qos)
        {
            return mqttClient.PublishAsync(topic, payload, qos);
        }
        private Task ConnectedHandler(MqttClientConnectedEventArgs
arg)
        {
            oConnectedEvent.Set();
            return Task.CompletedTask;
        }

        private static Task
DisconnectedHandler(MqttClientDisconnectedEventArgs arg)
        {
            Console.WriteLine($"Disconnected
mqtt.cloud.yandex.net.");
            return Task.CompletedTask;
        }

        private Task
DataHandler(MqttApplicationMessageReceivedEventArgs arg)
        {

            SubscribedData(arg.ApplicationMessage.Topic,
arg.ApplicationMessage.Payload);
            return Task.CompletedTask;
        }

        private static bool
CertificateValidationCallback(X509Certificate cert, X509Chain
chain, SslPolicyErrors errors, IMqttClientOptions opts)
        {
            try
            {
                if (errors == SslPolicyErrors.None)
                {
                    return true;
                }

                if (errors ==
SslPolicyErrors.RemoteCertificateChainErrors)
                {
                    chain.ChainPolicy.RevocationMode =
X509RevocationMode.NoCheck;
                    chain.ChainPolicy.VerificationFlags =
X509VerificationFlags.NoFlag;
                    chain.ChainPolicy.ExtraStore.Add(rootCrt);
```

```
                    chain.Build((X509Certificate2)rootCrt);
                    var res =
chain.ChainElements.Cast<X509ChainElement>().Any(a =>
a.Certificate.Thumbprint == rootCrt.Thumbprint);
                    return res;
                }
            }
            catch { }

            return false;
        }

    }
}
```

Приложение В

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Threading;
using System.Windows.Forms;
using YaClass;
namespace WindowsFormsApp5proverka
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            timer.Tick += new EventHandler(label1_Click);
            timer.Tick += new EventHandler(label11_Click);
            timer.Tick += new EventHandler(label16_Click);
            timer.Interval = 500;
            timer.Start();


        }
        System.Windows.Forms.Timer timer = new
System.Windows.Forms.Timer();
        public int state = 0; // состояние склада 0 - простой, 1 -
работа
        public string text = "";
        private const string DeviceID = "are18v6krffaq7o1mldk"; //
my cloud are7vm1sa8h0pr5duead
```

```
                                                                    //
are18v6krffaq7o1mldk semen cloud
        string[] words;
        // used for login-password authentication
        private const string RegistryID = "aresgcakub7pk92mbhej";
// my cloud areese9ohba65k0nrln9

// aresgcakub7pk92mbhej semen cloud
        private const string RegistryPassword = "Adminsklad2020";
        private const string DevicePassword = "Adminsklad2020";
        private YaClient devClient = new YaClient();
        public bool isconnected = false;


        string topic = YaClient.TopicName(DeviceID,
EntityType.Device, TopicType.Commands);



        private void Form1_Load(object sender, EventArgs e)
        {
           axMintController1.SetVirtualControllerLink(); //
установка связи с контроллером, использовать
SetUSBControllerLink() при подключении к ПЛК
           axMintController1.set_TriggerMode(0, 0);
           axMintController1.set_TriggerMode(1, 0);
           axMintController1.set_TriggerMode(2, 0);
            /* используется при работе на реальном ПЛК
            axMintController1.set_Comms(36, 1);
            axMintController1.set_Comms(1, 36);
            axMintController1.set_Comms(8, 1);
            axMintController1.set_Comms(14, 40000);
            axMintController1.set_Comms(39, 200);
            if (!axMintController1.MintExecuting)
             {
                 axMintController1.DoMintRun();
```

```
            }
            */

        }

        private void axMintController1_SerialReceiveEvent(object
sender, EventArgs e)
        {

        }

        private void button3_Click(object sender, EventArgs e)
        {
            axMintController1.set_MoveR(0, 1000);
        }

        private void button4_Click(object sender, EventArgs e)
        {
            axMintController1.set_MoveR(0, -1000);
        }

        private void button5_Click(object sender, EventArgs e)
        {
            axMintController1.set_MoveR(1, 1000);
        }

        private void button6_Click(object sender, EventArgs e)
        {
            axMintController1.set_MoveR(1, -1000);
        }

        private void button7_Click(object sender, EventArgs e)
```

```
        {
            axMintController1.set_MoveR(2, 1000);
        }

        private void button8_Click(object sender, EventArgs e)
        {
            axMintController1.set_MoveR(2, -1000);
        }

        async public void button9_Click(object sender, EventArgs
e)
        {
            await Task.Run(() => GoHome());
        }

        private void label1_Click(object sender, EventArgs e)
        {
            string s = "";
            s = axMintController1.get_Pos(0).ToString() + "\n" +
axMintController1.get_Pos(1).ToString() +
                "\n" + axMintController1.get_Pos(2).ToString();
            label1.Text = s;
        }
        private void button10_Click(object sender, EventArgs e)
        {
            if(axMintController1.get_DriveEnable(0) ||
axMintController1.get_DriveEnable(1)
                || axMintController1.get_DriveEnable(2) == false)
            {
                axMintController1.set_DriveEnable(0, true);
                axMintController1.set_DriveEnable(1, true);
                axMintController1.set_DriveEnable(2, true);
            }
```

```
        }

        public void GoHome()
        {   // вставить обнуление координат, тк работают с вирт
ПЛК
            state = 1;
            axMintController1.set_Pos(0, 0);
            axMintController1.set_Pos(1, -88181);
            axMintController1.set_Pos(2, 0);

            /* используется при работе на реально ПЛК
            axMintController1.set_Speed(0, 15000);
            axMintController1.set_Home(0, 2);
            while (axMintController1.get_AxisMode(0) != 0)
            {

            }
            axMintController1.set_Speed(1, 10000);
            axMintController1.set_Speed(2, 10000);
            axMintController1.set_Home(1, 2);
            axMintController1.set_Home(2, 2);
            while (axMintController1.get_AxisMode(1) != 0)
            {

            }
            axMintController1.set_MoveA(1, -88181);
            while (axMintController1.get_AxisMode(1) != 0)
            {

            }
            */
            state = 0;
```

```
        }

        private void comboBox1_SelectedIndexChanged(object sender,
EventArgs e)
        {
            if(comboBox1.SelectedItem.ToString() == "From
conveyor")
            {
                comboBox4.Enabled = true;
                comboBox5.Enabled = true;
                comboBox2.Enabled = false;
                comboBox3.Enabled = false;
                comboBox6.Enabled = false;
            }
            else if (comboBox1.SelectedItem.ToString() == "To
conveyor")
            {
                comboBox2.Enabled = true;
                comboBox3.Enabled = true;
                comboBox4.Enabled = false;
                comboBox5.Enabled = false;

            }
            else if (comboBox1.SelectedItem.ToString() == "Empty")
            {
                comboBox2.Enabled = true;
                comboBox3.Enabled = true;
                comboBox4.Enabled = true;
                comboBox5.Enabled = true;
                comboBox6.Enabled = true;
            }
        }
        // сначала всегда пишем столбец, потом строку
```

```
        public void Movement(string conveyor = "Empty", int
column1 = 0, int row1 = 0,
            int column2 = 0, int row2 = 0, string pillar = "Left")
        {
            state = 1;


            try
            {
                switch (conveyor)
                {
                    case "From conveyor":
                        axMintController1.set_MoveA(0, -495990);
                        axMintController1.set_MoveA(2, -130987);
                        while ((axMintController1.get_AxisMode(0)
+ axMintController1.get_AxisMode(2)) != 0)
                        {


                        }
                        axMintController1.set_MoveA(2, -144987);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {
                            //вставить сюда таймер
                        }
                        axMintController1.set_MoveA(2, -131987);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(0, -148989 -
51750 * (column2 - 1));
                        while (axMintController1.get_AxisMode(0)
!= 0)
```

```
                        {


                        }
                        axMintController1.set_MoveA(2, -23987 -
38997 * (row2 - 1) + 2400);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(1, -39173);
                        while (axMintController1.get_AxisMode(1)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(2, -23987 -
38997 * (row2 - 1) - 2400);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(1, -88181);
                        while (axMintController1.get_AxisMode(1)
!= 0)
                        {


                        }
                        break;
                    case "To conveyor":
                        axMintController1.set_MoveA(0, -148989 -
51750 * (column1 - 1));
```

```
                        while (axMintController1.get_AxisMode(0)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(2, -23987 -
38997 * (row1 - 1) - 2400);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(1, -136189);
                        while (axMintController1.get_AxisMode(1)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(2, -23987 -
38997 * (row1 - 1) + 2400);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(1, -88181);
                        while (axMintController1.get_AxisMode(1)
!= 0)
                        {


                        }
                        axMintController1.set_MoveA(0, -495990);
                        axMintController1.set_MoveA(2, -130987);
```

```
                        while ((axMintController1.get_AxisMode(0)
+ axMintController1.get_AxisMode(2)) != 0)
                        {


                        }
                        axMintController1.set_MoveA(2, -144987);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {
                            // вставить таймер
                        }
                        axMintController1.set_MoveA(2, -131987);
                        while (axMintController1.get_AxisMode(2)
!= 0)
                        {


                        }
                        break;
                    case "Empty":
                        if (pillar == "Left")
                        {
                            axMintController1.set_MoveA(0, -148989
- 51750 * (column1 - 1));
                            while
(axMintController1.get_AxisMode(0) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row1 - 1) - 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {
```

```
                            }
                            axMintController1.set_MoveA(1, -
39173);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row1 - 1) + 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
88181);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(0, -148989
- 51750 * (column2 - 1));
                            while
(axMintController1.get_AxisMode(0) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row2 - 1) + 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {
```

```
                            }
                            axMintController1.set_MoveA(1, -
39173);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row2 - 1) - 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
88181);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                        }
                        else if (pillar == "Right")
                        {
                            axMintController1.set_MoveA(0, -148989
- 51750 * (column1 - 1));
                            while
(axMintController1.get_AxisMode(0) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row1 - 1) - 2400);
```

```
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
136189);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row1 - 1) + 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
88181);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(0, -148989
- 51750 * (column2 - 1));
                            while
(axMintController1.get_AxisMode(0) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row2 - 1) + 2400);
```

```
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
137189);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                            axMintController1.set_MoveA(2, -23987
- 38997 * (row2 - 1) - 2400);
                            while
(axMintController1.get_AxisMode(2) != 0)
                            {


                            }
                            axMintController1.set_MoveA(1, -
88181);
                            while
(axMintController1.get_AxisMode(1) != 0)
                            {


                            }
                        }
                        break;
                    }
                }
                catch
                {
                    state = 0;
                }
```

```
            finally
            {
                state = 0;
            }

        }

        async public void button13_Click(object sender, EventArgs
e)
        {
            string conveyor = comboBox1.SelectedItem.ToString();
            int column1 = Convert.ToInt32(comboBox2.SelectedItem);
            int row1 = Convert.ToInt32(comboBox3.SelectedItem);
            int column2 = Convert.ToInt32(comboBox4.SelectedItem);
            int row2 = Convert.ToInt32(comboBox5.SelectedItem);
            string pillar = "";
            if (comboBox6.SelectedItem != null)
            {
                pillar = comboBox6.SelectedItem.ToString();
            }
            if (state == 1) { return; }
            await Task.Run(() => Movement(conveyor, column1, row1,
column2, row2, pillar));



        }

        private void comboBox2_SelectedIndexChanged(object sender,
EventArgs e)
        {


        }
```

```
        private void comboBox3_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void comboBox4_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void comboBox5_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void comboBox6_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void label2_Click(object sender, EventArgs e)
        {

        }

        private void label4_Click(object sender, EventArgs e)
        {

        }
```

```
        private void label5_Click(object sender, EventArgs e)
        {

        }

        private void comboBox7_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void label8_Click(object sender, EventArgs e)
        {

        }

        private void button1_Click(object sender, EventArgs e)
        {
            short axis =
short.Parse(comboBox7.SelectedItem.ToString());
            float value =
float.Parse(comboBox8.SelectedItem.ToString());
            axMintController1.set_Speed(axis, value);
        }

        private void comboBox8_SelectedIndexChanged(object sender,
EventArgs e)
        {

        }

        private void button11_Click(object sender, EventArgs e)
```

```
        {
            if (!isconnected)
            {
                devClient.Start(DeviceID, DevicePassword);
                devClient.SubscribedData += DataHandler;
                if (!devClient.WaitConnected())
                {
                    return;
                }
                devClient.Subscribe(topic,
MQTTnet.Protocol.MqttQualityOfServiceLevel.AtLeastOnce).Wait();
                isconnected = true;
            }

        }
        public bool validation(string str)
        { string[] values = { "1", "2", "3", "4", "5" };
            try
            {
                words = str.Split(' ');
                if (words[0] == "From" || words[0] == "To" ||
words[0] == "Empty")
                {
                    if (words[1] == "conveyor")
                    {
                        if(words[2] == "to" || words[2] == "from")
                        {
                            if(words[3] == "column")
                            {
                                if (values.Contains(words[4]))
                                {
```

```
                                    if(words[5] == "row")
                                    {
                                        if
(values.Contains(words[6]))
                                        {
                                            return true;
                                        }
                                        else { return false; }
                                    }
                                    else { return false; }
                                }
                                else { return false; }
                            }
                            else { return false; }
                        }

                        else { return false; }
                    }
                    else if (words[1] == "column")
                    {
                        if (values.Contains(words[2]))
                        {
                            if(words[3] == "row")
                            {
                                if (values.Contains(words[4]))
                                {
                                    if(words[5] == "to")
                                    {
                                        if(words[6] == "column")
                                        {
                                            if
(values.Contains(words[7]))
```

```
                                                    {
                                                        if (words[8] ==
"row")
                                                        {
                                                            if
(values.Contains(words[9]))
                                                            {
                                                                if
(words[10] == "pillar")
                                                                {
                                                                    if
(words[11] == "left" || words[11] == "right")
                                                                    {

return true;
                                                                    }
                                                                    else {
return false; }
                                                                }
                                                                else {
return false; }
                                                            }
                                                            else { return
false; }
                                                        }
                                                        else { return
false; }
                                                    }
                                                    else { return false; }
                                                }
                                                else { return false; }
                                            }
                                            else { return false; }
                                        }
                                        else { return false; }
```

```
                        }
                        else { return false; }
                    }
                    else { return false; }
                }
                else { return false; }
            }
            else { return false; }
        }
        catch
        {
            return false;
        }
        finally
        {


        }
    }
    async public void DataHandler(string topic, byte[]
payload)
    {
        text = System.Text.Encoding.UTF8.GetString(payload);
        if(state == 1) { return; }
        if (validation(text))
        {   if(words[0] == "To")
            {
                string conveyor = words[0] + ' ' + words[1];
                int column1 = Convert.ToInt32(words[4]);
                int row1 = Convert.ToInt32(words[6]);
                await Task.Run(() =>
Movement(conveyor,column1,row1));
            }
```

```
                else if (words[0] == "From")
                {
                    string conveyor = words[0] + ' ' + words[1];
                    int column2 = Convert.ToInt32(words[4]);
                    int row2 = Convert.ToInt32(words[6]);
                    await Task.Run(() =>
Movement(conveyor,0,0,column2,row2));
                }
                else if (words[0] == "Empty")
                {
                    string conveyor = words[0];
                    int column1 = Convert.ToInt32(words[2]);
                    int row1 = Convert.ToInt32(words[4]);
                    int column2 = Convert.ToInt32(words[7]);
                    int row2 = Convert.ToInt32(words[9]);
                    string pillar = words[11];
                    await Task.Run(() =>
Movement(conveyor,column1,row1,column2,row2,pillar));
                }

            }

        }


        public void label11_Click(object sender, EventArgs e)
        {
            label11.Text = text;
        }

        private void button12_Click(object sender, EventArgs e)
        {
```

```
            if (isconnected)
            {
                string payload = textBox1.Text;

devClient.Publish("$devices/are18v6krffaq7o1mldk/events/test",
payload, MQTTnet.Protocol.MqttQualityOfServiceLevel.AtLeastOnce);
            }

        }

        private void button2_Click_1(object sender, EventArgs e)
        {
            devClient.Stop();
            isconnected = false;
        }

        private void textBox1_TextChanged(object sender, EventArgs
e)
        {

        }

        private void label12_Click(object sender, EventArgs e)
        {

        }

        private void label13_Click(object sender, EventArgs e)
        {

        }

        private void button14_Click(object sender, EventArgs e)
```

```
        {
            if (axMintController1.get_DriveEnable(0) ||
axMintController1.get_DriveEnable(1)
                || axMintController1.get_DriveEnable(2) == true){
                axMintController1.set_DriveEnable(0, false);
                axMintController1.set_DriveEnable(1, false);
                axMintController1.set_DriveEnable(2, false);
            }
        }

        private void label16_Click(object sender, EventArgs e)
        {   if(axMintController1.get_DriveEnable(0) &&
axMintController1.get_DriveEnable(1)
                && axMintController1.get_DriveEnable(2) == true) {
label16.Text = "Drivers state: ON"; }
            else { label16.Text = "Drivers state: OFF"; }
        }
    }
}
```

Приложение Г

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

Курсовой проект на тему:

«Проектирование программной части сервера устройства для облачной системы управления складом на основе облачной платформы «YandexCloud»»

Выполнил студен РК9-74Б:

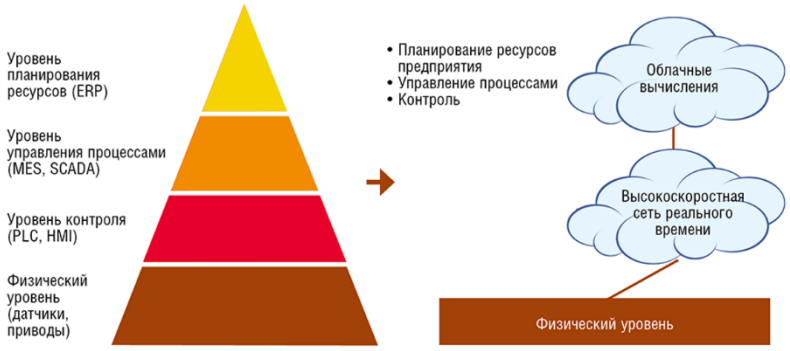
Масный Д.И.

Руководитель проекта:

Буханов С.А.

2020

Анализ предметной области



2

Цель и задачи

Цель данного курсового проекта: создание программной части сервера устройства для облачной системы управления модулем транспортно-накопительной системы (складом) на основе облачной платформы «Yandex Cloud».

В ходе работы над проектом были поставлены следующие **задачи**:

1) проанализировать существующее ПО, управляющее складом и выявить его недостатки;

2) на основе анализа полученных данных сформулировать требования, предъявляемые к проектируемой программной части сервера устройства;

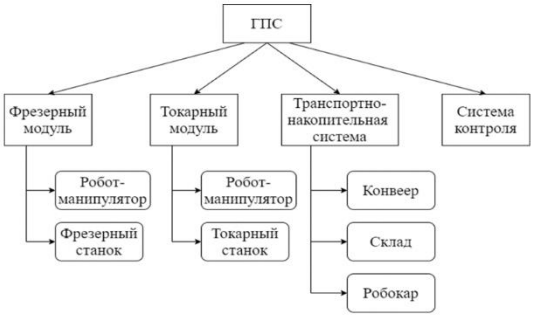
3) разработать функциональную модель перемещения паллет складом;

4) разработать алгоритмы основных функций программы;

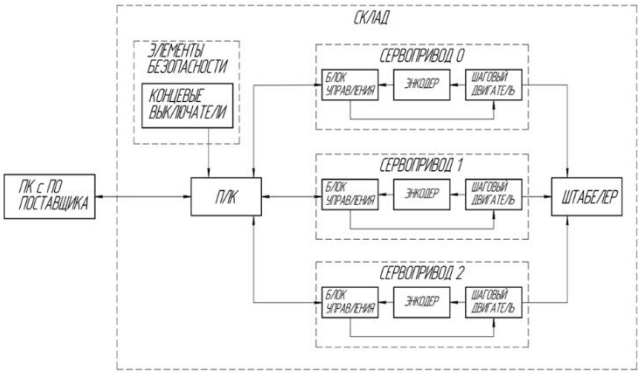
5) разработать исходный код программной части сервера устройства на языке C#.

3

Анализ существующей системы



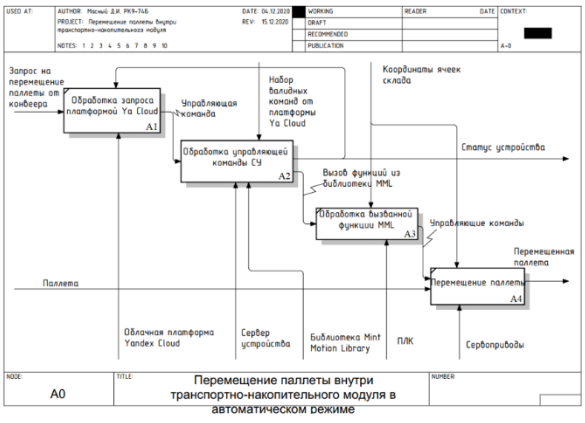
Структурная схема ГПС «Denford»



Структурная схема склада

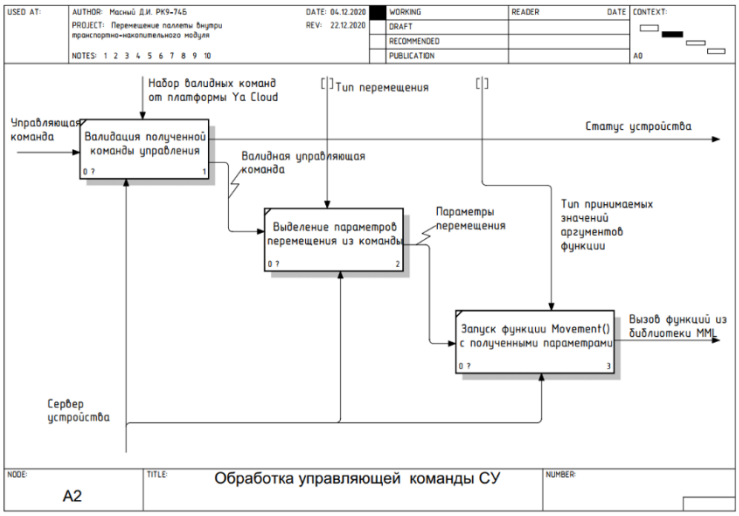
4

Функциональная модель перемещения паллет складом



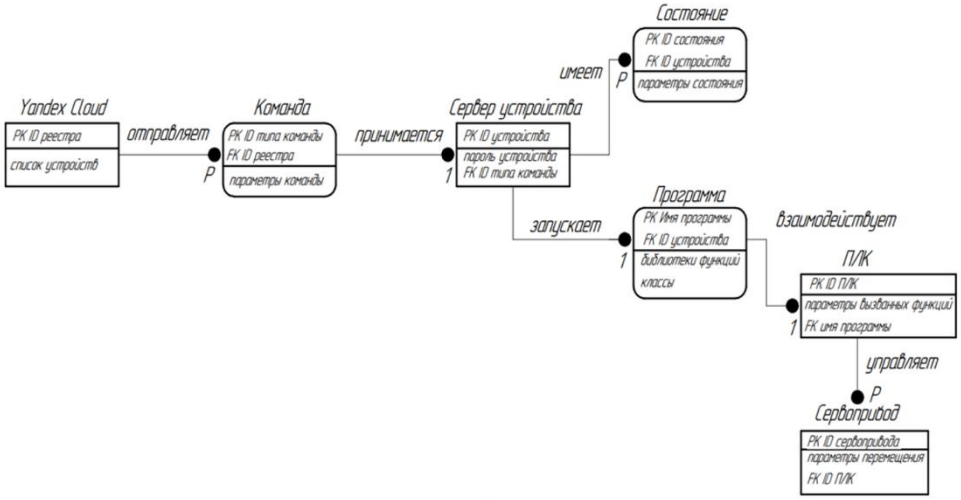
5

Функциональная модель перемещения паллет складом



6

Концептуальная модель предметной области



7

Взаимодействие сервера устройства с облачной платформой «Yandex Cloud»

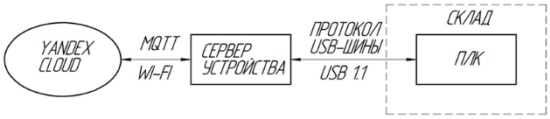
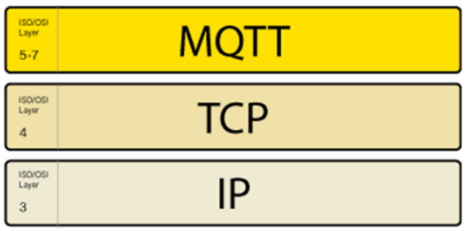
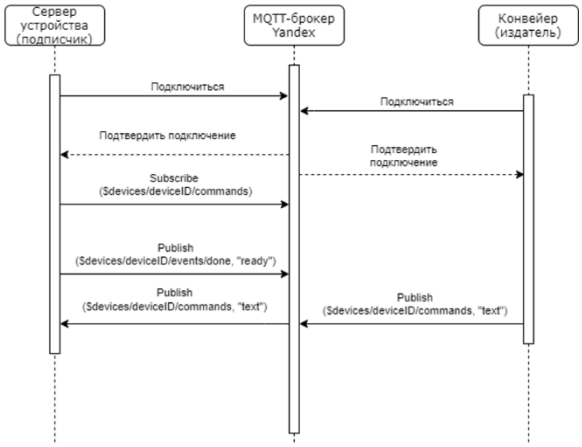


Схема взаимодействия устройств



Уровни протоколов передачи данных



8 UML-диаграмма последовательности взаимодействия сервера устройства с облаком

Взаимодействие сервера устройства с ПЛК склада

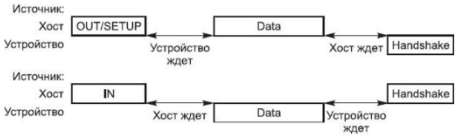
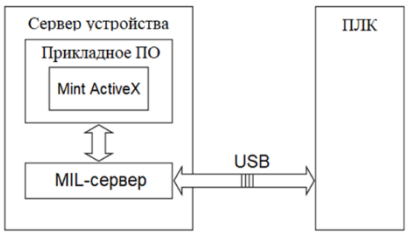
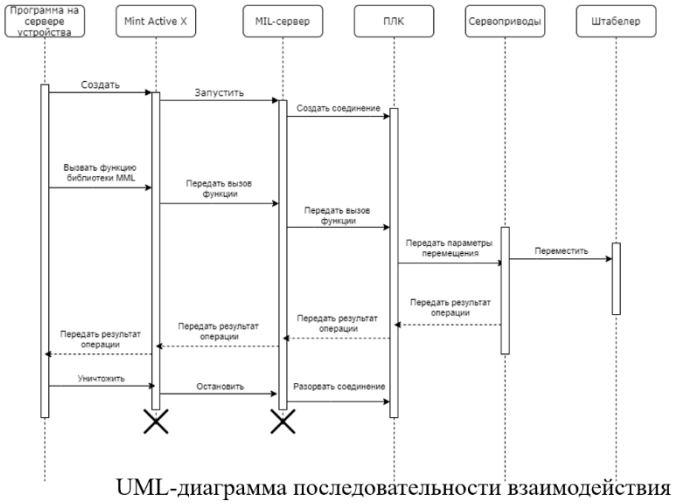


Схема передачи данных по протоколу USB

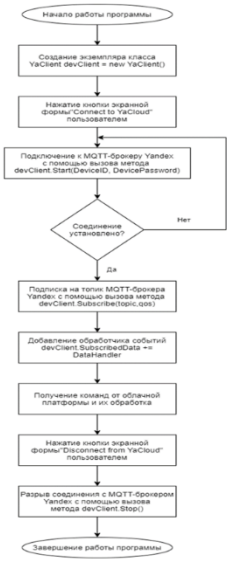


Структурная схема взаимодействия сервера устройства с ПЛК

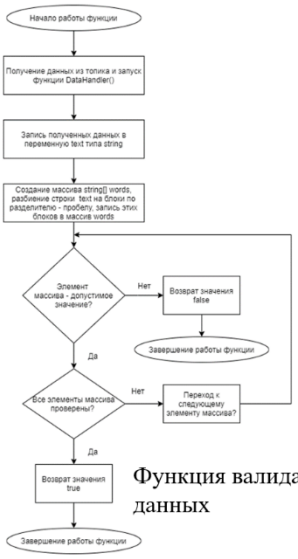


UML-диаграмма последовательности взаимодействия сервера устройства с ПЛК

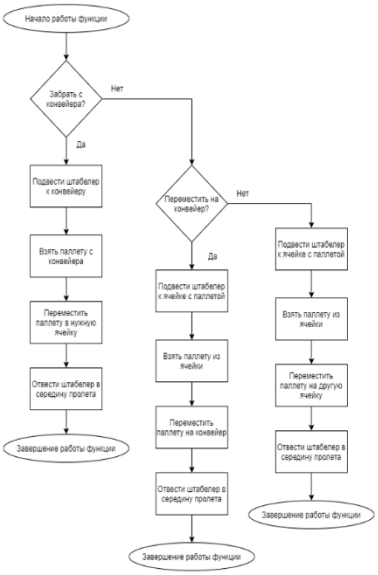
Алгоритмы основных функций программы



Функция подключения к облаку

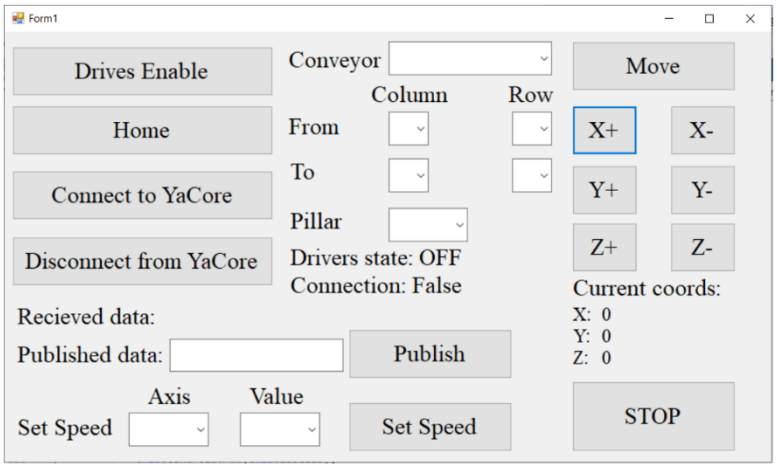


Функция валидации данных



Функция перемещения паллет

Разработка графического интерфейса программы



11

Заключение

1) В настоящее время внедряются кибер-физические системы и технологии промышленного интернета вещей в производство. Это приводит к переходу от традиционных АСУ ТП к «облачным», в которых планирование ресурсов предприятия, управление процессами и контроль осуществляется в облачной платформе.

2) Анализ существующего программного обеспечения, управляющего складским модулем, показал, что данное ПО имеет ряд недостатков, основным из которых является невозможность расширения производства.

3) Были сформированы требования к разрабатываемой системе, важнейшее из которых – возможность автономной работы складского модуля при подключении к облачной платформе «Yandex Cloud».

12

Заключение

4) Были разработаны: функциональная модель перемещения паллет складом, концептуальная модель взаимодействия облачной платформы, сервера устройства и ПЛК складского модуля.

5) Были разработаны алгоритмы: функции подключения сервера устройства к облачной платформе, функции валидации полученных команд от платформы «Yandex Cloud», функции перемещения штабелера склада.

6) Был разработан исходный код программной части сервера устройства на языке C#.

Разработанная программная часть сервера устройства совместно с облачной платформой «Yandex Cloud» реализуют концепцию промышленного интернета вещей в рамках рассматриваемого складского модуля ГПС «Denford».

Программа позволяет оператору взаимодействовать со складом через интуитивно понятную экранную форму по одному из двух сценариев: 1) «автономный» - команды на перемещение паллет передаются серверу устройства по беспроводной сети через протокол MQTT от облачной платформы «Yandex Cloud»; 2) «режим управления оператором» - управление складом осуществляется непосредственно оператором через сервер устройства путем заполнения соответствующих полей экранной формы. Выбор сценария осуществляет оператор ГПС в зависимости от его целей и задач.